

Otimização de Modelos de Aprendizado Federado com Redes Definidas por Software

Cândido Leandro de Queiroga Bisneto



CENTRO DE INFORMÁTICA
UNIVERSIDADE FEDERAL DA PARAÍBA

João Pessoa, 2025

Cândido Leandro de Queiroga Bisneto

Otimização de Modelos de Aprendizado Federado com Redes Definidas por Software

Monografia apresentada ao curso Ciência da Computação
do Centro de Informática, da Universidade Federal da Paraíba,
como requisito para a obtenção do grau de Bacharel em Ciência da Computação

Orientador: Fernando Menezes Matos

Dezembro de 2025



CENTRO DE INFORMÁTICA
UNIVERSIDADE FEDERAL DA PARAÍBA

Trabalho de Conclusão de Curso de Ciência da Computação intitulado ***Otimização de Modelos de Aprendizado Federado com Redes Definidas por Software*** de autoria de Cândido Leandro de Queiroga Bisneto, aprovada pela banca examinadora constituída pelos seguintes professores:

Prof. Dr. Fernando Menezes Matos
Universidade Federal da Paraíba

Prof. [Nome do Professor B]
Universidade Federal da Paraíba

Prof. [Nome do Professor C]
Universidade Federal da Paraíba

Coordenador(a) do Curso de Ciência da Computação
[Nome do Coordenador]
CI/UFPB

João Pessoa, Dezembro de 2025

DEDICATÓRIA

[Texto da dedicatória - opcional]

AGRADECIMENTOS

[Texto dos agradecimentos]

RESUMO

Este trabalho investiga o desempenho de três algoritmos de *Gradient Boosting* baseados em árvore — XGBoost, LightGBM e CatBoost — em ambiente de Aprendizado Federado aplicado à detecção de fraude em abertura de contas bancárias. Utilizando o *dataset* BAF (*Bank Account Fraud*) e o *framework* Flower para simulação federada, foram avaliadas duas estratégias de agregação — Bagging e Cíclica — com 3 clientes, 5 rodadas de comunicação e particionamento IID balanceado. Os modelos federados alcançaram TPR entre 54,83% e 56,79% no ponto de operação de 5% de FPR, resultados competitivos com o *baseline* centralizado do BAF (40–55%). O XGBoost Bagging obteve o melhor desempenho absoluto (56,79% TPR, AUC 0,8934), enquanto o CatBoost Cíclico alcançou resultado praticamente equivalente (56,72% TPR) com menor custo de comunicação (1,27 MB). A aplicação de limiares independentes por grupo demográfico elevou o FPR Ratio de $\approx 0,3$ para $\approx 0,88$ –1,0, demonstrando que a equidade algorítmica pode ser alcançada com perda média de apenas 2,0 pontos percentuais em TPR. Os resultados demonstram a viabilidade de modelos baseados em árvore para sistemas federados de detecção de fraude, oferecendo eficiência de comunicação superior a abordagens baseadas em redes neurais profundas.

Palavras-chave: Aprendizado Federado, Detecção de Fraudes Bancárias, XGBoost, LightGBM, CatBoost, Flower Framework, Privacidade de Dados, Modelos Baseados em Árvore.

ABSTRACT

This work investigates the performance of three tree-based *Gradient Boosting* algorithms — XGBoost, LightGBM, and CatBoost — in a Federated Learning environment applied to bank account opening fraud detection. Using the BAF (Bank Account Fraud) dataset and the Flower framework for federated simulation, two aggregation strategies — Bagging and Cyclic — were evaluated with 3 clients, 5 communication rounds, and balanced IID partitioning. The federated models achieved TPR between 54.83% and 56.79% at the 5% FPR operating point, results competitive with the centralized BAF baseline (40–55%). XGBoost Bagging achieved the best absolute performance (56.79% TPR, AUC 0.8934), while CatBoost Cyclic reached a nearly equivalent result (56.72% TPR) with lower communication cost (1.27 MB). Applying independent thresholds per demographic group raised the FPR Ratio from ≈ 0.3 to ≈ 0.88 –1.0, demonstrating that algorithmic fairness can be achieved with an average loss of only 2.0 percentage points in TPR. The results demonstrate the viability of tree-based models for federated fraud detection systems, offering superior communication efficiency compared to deep neural network approaches.

Keywords: Federated Learning, Bank Fraud Detection, XGBoost, LightGBM, CatBoost, Flower Framework, Data Privacy, Tree-Based Models.

LISTA DE FIGURAS

1	Inicialização do modelo global no servidor central. Fonte: [?]	28
2	Distribuição do modelo global para os clientes. Fonte: [?]	29
3	Treinamento local nos clientes com dados privados. Fonte: [?]	30
4	Envio das atualizações locais para o servidor. Fonte: [?]	31
5	Agregação das atualizações no servidor. Fonte: [?]	32
6	Arquitetura do Flower: framework agnóstico que suporta diversos tipos de clientes e algoritmos de ML. Fonte: [?]	40
7	AUC-ROC por modelo e estratégia. Valores calculados no conjunto de teste (mês 7).	62
8	<i>Trade-off</i> entre performance (TPR) e equidade (FPR Ratio). Círculos: limiar único; quadrados: limiar por grupo. A região verde indica fairness aceitável ($\geq 0,9$).	63
9	Convergência do TPR@5%FPR ao longo das 15 rodadas. Painel esquerdo: limiar único. Painel direito: limiar por grupo. Linha pontilhada vermelha: <i>baseline</i> centralizado (55%).	64
10	Estratégia Cíclica: contribuição sequencial de cada cliente. A cor indica qual cliente treinou em cada rodada. A linha cinza conecta os pontos para mostrar a tendência.	66
11	Tempo de execução por modelo e estratégia. A estratégia Cíclica é consistentemente mais rápida.	67
12	Volume total de dados transferidos por modelo e estratégia. O CatBoost Cíclico é o mais eficiente (9,62 MB).	68

LISTA DE TABELAS

1	Análise comparativa dos trabalhos relacionados	25
2	Tecnologias utilizadas no projeto	47
3	Resumo das variantes do dataset BAF. Valores entre parênteses referem-se ao conjunto de teste. Adaptado de [?].	49
4	Metadados do dataset BAF Base utilizado nos experimentos	50
5	Features do dataset BAF	52
6	Especificações do ambiente computacional	57
7	Configuração do dataset para os experimentos	57
8	Hiperparâmetros otimizados para cada algoritmo	58
9	Parâmetros da simulação federada	59
10	Experimentos realizados	59
11	Resultados finais no conjunto de teste (mês 7). FPR Ratio = min / max das FPR entre grupos demográficos; 1,0 indica fairness perfeita. Melhores valores em negrito.	61
12	Evolução do TPR (limiar standard) no conjunto de validação por rodada selecionada	63
13	Tempo de execução e volume de comunicação das simulações federadas (15 rodadas)	67

LISTA DE ABREVIATURAS

API - Application Programming Interface

AUC - Area Under the Curve

CART - Classification and Regression Trees

CPU - Central Processing Unit

DP - Differential Privacy

FL - Federated Learning

GBDT - Gradient Boosting Decision Trees

GPU - Graphics Processing Unit

gRPC - Google Remote Procedure Call

HTTP - Hypertext Transfer Protocol

IID - Independent and Identically Distributed

LGPD - Lei Geral de Proteção de Dados

ML - Machine Learning

non-IID - Non-Independent and Identically Distributed

QoS - Quality of Service

ROC - Receiver Operating Characteristic

SDN - Software-Defined Networking

TCC - Trabalho de Conclusão de Curso

TCP - Transmission Control Protocol

UFPB - Universidade Federal da Paraíba

UDP - User Datagram Protocol

Sumário

1	INTRODUÇÃO	17
1.1	Motivação	17
1.2	Objetivo	18
1.3	Estrutura da monografia	20
2	REVISÃO DE LITERATURA	21
2.1	Evolução do Aprendizado Federado	21
2.2	Modelos Baseados em Árvore em Ambientes Federados	22
2.3	Detecção de Fraudes Bancárias e Aprendizado Federado	23
2.4	Heterogeneidade de Dados e Fairness	23
2.5	Privacidade e Segurança	24
2.6	Análise Comparativa e Posicionamento do Trabalho	25
3	FUNDAMENTOS TEÓRICOS	27
3.1	Aprendizado Federado	27
3.1.1	Definição e Características	27
3.1.2	Estratégias de Agregação	34
3.2	Modelos Baseados em Árvore de Decisão	35
3.2.1	XGBoost	36
3.2.2	LightGBM	37
3.2.3	CatBoost	38
3.3	Framework Flower	39
3.3.1	Arquitetura do Flower	40
4	METODOLOGIA	43
4.1	Visão Geral da Solução Proposta	43
4.2	Arquitetura do Sistema	43
4.3	Particionamento dos Dados entre Clientes	44
4.4	Implementação dos Modelos Federados	44

4.5	Estratégias de Agregação	45
4.6	Métricas de Avaliação	46
4.7	Tecnologias Utilizadas	47
4.8	Limitações Metodológicas	47
5	DATASET E PREPARAÇÃO DOS DADOS	48
5.1	O Dataset Bank Account Fraud (BAF)	48
5.1.1	Contexto e Domínio de Aplicação	48
5.1.2	Geração dos Dados e Preservação de Privacidade	48
5.1.3	Suite de Datasets e Variantes	49
5.2	Estrutura e Características do Dataset	50
5.3	Descrição das Features	51
5.4	Desbalanceamento de Classes	53
5.5	Estratégia de Divisão Temporal	53
5.6	Pré-processamento	54
5.6.1	Limpeza dos Dados	54
5.6.2	Engenharia de Features	54
5.6.3	Codificação e Preparação Final	55
5.7	Particionamento dos Dados para Federated Learning	55
5.7.1	Estratégia de Particionamento IID Balanceado	55
6	CONFIGURAÇÃO EXPERIMENTAL	57
6.1	Ambiente Computacional	57
6.2	Configuração do Dataset	57
6.3	Otimização Federada de Hiperparâmetros	58
6.4	Configuração do Aprendizado Federado	58
6.5	Experimentos Realizados	59
6.6	Métricas de Avaliação	60
7	RESULTADOS	61
7.1	Baseline Centralizado	61

7.2	Resultados Federados	61
7.3	Convergência	63
7.4	Contribuição Sequencial dos Clientes (Estratégia Cíclica)	65
7.5	Eficiência	67
7.6	Comparação Federado vs. Centralizado	69
8	DISCUSSÃO	70
8.1	Interpretação dos Resultados	70
8.2	Comparação com Estado da Arte	70
8.3	Desafios de Implementação	71
8.4	Trade-off entre Fairness e Desempenho	73
8.5	Limitações do Trabalho	73
8.6	Implicações Práticas	74
9	CONCLUSÕES E TRABALHOS FUTUROS	76
9.1	Conclusões	76
9.2	Contribuições	76
9.3	Trabalhos Futuros	76
	REFERÊNCIAS	76

1 INTRODUÇÃO

1.1 Motivação

A detecção de fraudes bancárias constitui um dos desafios mais críticos enfrentados por instituições financeiras na era digital. O crescimento exponencial de transações online, aliado à sofisticação crescente dos métodos empregados por fraudadores, torna os sistemas tradicionais baseados em regras progressivamente insuficientes para acompanhar a dinâmica e complexidade das tentativas de fraude [?]. De acordo com a literatura recente, uma queda de poucos pontos percentuais no desempenho preditivo de um modelo de detecção pode representar milhões em perdas financeiras para as instituições, ao passo que falsos positivos — clientes legítimos incorretamente classificados como fraudulentos — geram insatisfação, perda de clientes e danos reputacionais significativos [?].

Historicamente, a detecção de fraudes tem dependido de arquiteturas centralizadas, onde dados de transações são agregados em um único servidor para treinamento de modelos [?]. Contudo, esta abordagem enfrenta desafios crescentes relacionados à privacidade dos dados e regulamentações rigorosas como o GDPR (*General Data Protection Regulation*) na União Europeia e a LGPD (Lei Geral de Proteção de Dados) no Brasil [?, ?]. Segundo a análise de Farrukh et al. [?], embora os métodos de *Machine Learning* tradicionais ofereçam elevada precisão, eles criam silos de dados e pontos únicos de falha que comprometem a segurança da informação sensível dos clientes.

O Aprendizado Federado (*Federated Learning* — FL) surge como resposta a este dilema, representando uma evolução necessária do aprendizado de máquina em resposta às regulamentações de privacidade cada vez mais rigorosas [?, ?]. Este paradigma permite que múltiplas instituições financeiras colaborem no treinamento de modelos de detecção de fraude sem compartilhar dados brutos, resolvendo o impasse entre utilidade do modelo e privacidade do usuário [?]. No FL, cada participante treina o modelo localmente utilizando seus próprios dados, compartilhando apenas as atualizações do modelo com um servidor central que coordena a agregação [?]. Segundo Kennedy et al. [?], a colaboração *Cross-Silo* entre bancos via FL permite reduzir a taxa de falsos positivos em até 15% comparado com modelos isolados, validando a abordagem de treinamento colaborativo.

Dentre os diversos algoritmos de *Machine Learning* disponíveis, modelos baseados em árvore de decisão destacam-se como estado da arte para dados tabulares. Especificamente, XGBoost [?], LightGBM [?] e CatBoost [?] são amplamente utilizados tanto em competições de ciência de dados quanto em sistemas de produção no setor financeiro. Ao contrário de abordagens baseadas em redes neurais profundas, que exigem alto custo computacional e comunicação intensiva [?], estes modelos oferecem um compromisso ideal entre eficiência e acurácia, sendo particularmente adequados para ambientes com recursos

limitados [?, ?].

A escolha por modelos de *Gradient Boosting* em ambientes federados é validada pela literatura recente. Cheng et al. [?] demonstraram com o SecureBoost que é possível manter precisão comparável ao treinamento centralizado (cerca de 90% de acurácia em benchmarks de fraude) enquanto preserva-se a privacidade. Li et al. [?] mostraram com o FedTree que sistemas de árvore otimizados podem ser ordens de magnitude mais rápidos que abordagens ingênuas, aspecto crucial para detecção de fraude em tempo real. Na taxonomia proposta por Liu et al. [?], a eficiência de comunicação inerente aos modelos baseados em árvore, que utilizam histogramas e discretização, alinha-se com os princípios de otimização de largura de banda defendidos para sistemas FL em escala [?].

A combinação de modelos baseados em árvore com Aprendizado Federado para detecção de fraudes bancárias apresenta vantagens significativas. Primeiramente, modelos como XGBoost, LightGBM e CatBoost são computacionalmente mais leves que redes neurais profundas, não exigindo GPUs dedicadas para treinamento, o que viabiliza a participação de clientes com recursos limitados [?]. Em segundo lugar, estes modelos são nativamente adequados para dados tabulares, que constituem a forma predominante de dados no setor financeiro. Em terceiro lugar, a natureza baseada em *ensemble* destes algoritmos permite estratégias de agregação federada específicas, como Bagging e Cíclica, que exploram a composição aditiva de árvores para combinar conhecimento de múltiplos participantes de forma eficiente.

1.2 Objetivo

Este trabalho tem como objetivo principal investigar o desempenho de modelos baseados em árvore — especificamente XGBoost, LightGBM e CatBoost — em ambientes de Aprendizado Federado aplicados à tarefa de detecção de fraude em abertura de contas bancárias. A escolha destes três algoritmos justifica-se por suas diferentes abordagens ao problema de *Gradient Boosting*, cada um com características particulares que podem apresentar comportamentos distintos em ambientes distribuídos, e por serem os modelos mais amplamente utilizados em sistemas de detecção de fraudes bancárias na indústria [?, ?].

Este estudo utiliza o dataset *Bank Account Fraud* (BAF) [?], publicado no NeurIPS 2022 (*Track on Datasets and Benchmarks*), que constitui o primeiro conjunto de dados publicamente disponível, de larga escala e com preservação de privacidade, voltado para detecção de fraudes em abertura de contas bancárias. O artigo original que introduz o BAF utiliza o LightGBM como algoritmo base para avaliar desempenho e *fairness* nos diferentes variantes do dataset. Este trabalho estende essa investigação ao contexto federado, avaliando não apenas o LightGBM, mas também o XGBoost e o CatBoost —

este último não explorado no artigo original — de modo a fornecer uma análise comparativa completa dos três principais algoritmos de *Gradient Boosting* baseados em árvore no cenário de detecção de fraude federada.

Um aspecto central deste trabalho é a avaliação do desempenho dos modelos federados em um cenário controlado com particionamento IID (*Independent and Identically Distributed*) balanceado entre os clientes. Embora a literatura documente extensivamente os desafios de dados heterogêneos (Non-IID) em sistemas federados [?, ?, ?], este trabalho adota o particionamento IID como *baseline* experimental, de modo a isolar o impacto do treinamento distribuído em si, sem a influência adicional da heterogeneidade estatística entre clientes. Esta abordagem permite quantificar com maior precisão o “custo da privacidade” introduzido pelo paradigma federado, fornecendo uma base comparativa sólida em relação aos resultados centralizados do artigo original do BAF [?].

Os objetivos específicos desta pesquisa são:

1. **Avaliar o desempenho dos modelos em ambiente federado:** Investigar como XGBoost, LightGBM e CatBoost se comportam quando treinados de forma distribuída utilizando o framework Flower, comparando suas métricas de classificação (acurácia, precisão, recall, F1-score, AUC) em múltiplas rodadas de comunicação.
2. **Comparar estratégias de agregação:** Analisar o impacto das estratégias de agregação Bagging e Cíclica no desempenho dos três algoritmos, avaliando qual combinação de algoritmo e estratégia produz os melhores resultados para o domínio de fraude bancária.
3. **Avaliar o impacto do particionamento federado:** Investigar como a distribuição IID balanceada dos dados entre clientes afeta o desempenho dos modelos em comparação com o treinamento centralizado, quantificando o custo introduzido pela descentralização dos dados em termos de métricas de classificação.
4. **Comparar com resultados centralizados:** Utilizar os resultados publicados no artigo do BAF como *baseline* comparativo, analisando a perda de desempenho (se houver) introduzida pelo treinamento federado em relação ao treinamento centralizado tradicional, quantificando assim o “custo da privacidade” [?].
5. **Analisar fairness em ambiente federado:** Avaliar métricas de justiça algorítmica (*fairness*) nos modelos federados, utilizando a métrica de *FPR Ratio* (razão entre taxas de falso positivo de diferentes grupos demográficos), conforme definido pelos autores do dataset BAF [?], para análise de viés demográfico em dois regimes de limiar de decisão.

A hipótese central que orienta esta pesquisa é que modelos baseados em árvore de decisão, devido à sua eficiência computacional e adequação a dados tabulares, são candidatos viáveis e eficazes para implementação em sistemas de Aprendizado Federado voltados à detecção de fraudes bancárias, mantendo desempenho competitivo em relação a abordagens centralizadas [?, ?]. Além disso, espera-se que as diferentes características dos três algoritmos resultem em comportamentos distintos no ambiente federado, fornecendo insights práticos para a escolha do algoritmo mais adequado em cenários reais de detecção de fraude distribuída.

1.3 Estrutura da monografia

Esta monografia está dividida da seguinte maneira: na Seção 2 é feita a revisão da literatura sobre Aprendizado Federado e detecção de fraudes bancárias com modelos baseados em árvore. Na Seção 3 são apresentados os fundamentos teóricos necessários para compreensão do trabalho. Na Seção 4 é apresentada a metodologia utilizada no desenvolvimento do trabalho. Na Seção 5 o dataset utilizado é descrito em detalhes. Na Seção 6 a configuração experimental é apresentada. Na Seção 7 os resultados obtidos são apresentados e analisados. Na Seção 8 é feita a discussão dos resultados. Na Seção 9 é feita a conclusão deste trabalho e trabalhos futuros são propostos.

2 REVISÃO DE LITERATURA

Esta seção apresenta uma revisão sistemática da literatura relacionada ao Aprendizado Federado aplicado à detecção de fraudes bancárias. A revisão está organizada em quatro subseções temáticas: trabalhos seminais sobre Aprendizado Federado, estudos sobre modelos baseados em árvore em ambientes federados, pesquisas específicas sobre detecção de fraudes bancárias, e uma análise comparativa que posiciona o presente trabalho em relação ao estado da arte.

2.1 Evolução do Aprendizado Federado

O Aprendizado Federado foi formalmente introduzido por McMahan et al. [?] com o algoritmo FedAvg (*Federated Averaging*), que estabeleceu os fundamentos para o treinamento distribuído de modelos de aprendizado de máquina preservando a privacidade dos dados. Neste trabalho seminal, os autores demonstraram que é possível treinar modelos de alta qualidade em dados distribuídos através da agregação de atualizações locais, sem a necessidade de centralizar os dados brutos. Esta contribuição foi posteriormente expandida por Yang et al. [?], que sistematizaram a taxonomia do campo, distinguindo entre Aprendizado Federado Horizontal (HFL), onde os participantes compartilham o mesmo espaço de atributos mas possuem amostras diferentes, e Aprendizado Federado Vertical (VFL), onde os participantes possuem atributos diferentes para as mesmas entidades.

A escalabilidade do Aprendizado Federado foi abordada por Bonawitz et al. [?], que propuseram protocolos para sistemas com milhões de dispositivos participantes. Os autores estabeleceram métricas fundamentais para avaliação de sistemas FL em escala, incluindo latência de comunicação, tolerância a falhas e eficiência de recursos. Paralelamente, Konečný et al. [?] investigaram estratégias para redução do custo de comunicação, um dos principais gargalos em sistemas federados, propondo técnicas de compressão e quantização de gradientes que permanecem relevantes para implementações práticas.

A evolução recente do campo foi documentada em surveys abrangentes. Liu et al. [?] apresentaram uma taxonomia atualizada que categoriza as abordagens de FL em dimensões de otimização, privacidade e aplicações, identificando direções abertas como personalização e eficiência. Chai et al. [?] focaram especificamente em metodologias de avaliação, argumentando que sistemas FL devem ser avaliados não apenas pela utilidade (performance do modelo), mas também pela eficiência e segurança. Shi [?] documentou a transição do processamento centralizado para abordagens distribuídas, destacando que o FL representa uma resposta às regulamentações de privacidade e às limitações práticas de transferência de grandes volumes de dados.

A transição do aprendizado centralizado para o distribuído foi analisada siste-

maticamente por Abdulrahman et al. [?], que traçaram a evolução histórica desde o processamento em mainframes até o Aprendizado Federado contemporâneo. Os autores argumentam que esta evolução não é apenas técnica, mas também regulatória, impulsionada por legislações como o GDPR que restringem a centralização de dados pessoais [?].

2.2 Modelos Baseados em Árvore em Ambientes Federados

A adaptação de algoritmos de *Gradient Boosting* para ambientes federados representa uma área de pesquisa ativa e particularmente relevante para aplicações em dados tabulares. Cheng et al. [?] introduziram o SecureBoost, um framework que permite o treinamento federado de modelos XGBoost sem perda de precisão em relação ao treinamento centralizado. O SecureBoost utiliza agregação segura para evitar o compartilhamento de gradientes brutos, mitigando riscos de vazamento de informação enquanto mantém acurácia de aproximadamente 90% em benchmarks de fraude. Esta contribuição é fundamental para o presente trabalho, pois valida a viabilidade de modelos baseados em árvore em cenários federados com requisitos rigorosos de privacidade.

Li et al. [?] propuseram o FedTree, um sistema otimizado para treinamento federado de árvores de decisão que demonstra ganhos significativos de eficiência. Os autores mostraram que sistemas de árvore otimizados podem ser ordens de magnitude mais rápidos que implementações ingênuas, um aspecto crucial para aplicações de detecção de fraude em tempo real onde a latência é crítica. O trabalho de Li, Wen e He [?] sobre Gradient Boosting Decision Trees federadas práticas complementa esta linha, demonstrando que é possível manter a eficiência computacional característica do LightGBM em ambientes distribuídos.

A questão da comunicação eficiente foi abordada por Le et al. [?] com o FedXGBoost, uma implementação que utiliza agregação segura para contornar a necessidade de compartilhar gradientes brutos. Os autores demonstraram que a serialização baseada em histogramas, inerente a modelos como LightGBM, alinha-se naturalmente com os princípios de quantização defendidos para sistemas FL eficientes [?]. Liu et al. [?] exploraram uma abordagem alternativa com o Federated Forest, utilizando encriptação para garantir privacidade em florestas de decisão, embora com maior overhead computacional.

Um diferencial importante dos modelos baseados em árvore em relação a redes neurais profundas é sua resistência natural a ataques de vazamento de informação. Zhu, Liu e Han [?] demonstraram que é possível recuperar dados de treinamento a partir de gradientes em redes neurais (*Deep Leakage from Gradients*). Contudo, conforme argumentado por Liu et al. [?], modelos baseados em árvore oferecem uma barreira natural adicional, pois suas atualizações são baseadas em histogramas e divisões (*splits*), tornando

a reconstrução exata dos dados tabulares significativamente mais difícil.

2.3 Detecção de Fraudes Bancárias e Aprendizado Federado

A aplicação de Aprendizado Federado à detecção de fraudes financeiras tem recebido atenção crescente na literatura recente. Farrukh et al. [?] apresentaram uma revisão sistemática comparando abordagens de Machine Learning centralizado com Federated Learning para detecção de fraudes, concluindo que o FL representa a evolução necessária para o setor bancário. Os autores destacam que, embora métodos centralizados ofereçam alta precisão, eles criam silos de dados e pontos únicos de falha incompatíveis com as exigências regulatórias contemporâneas.

Kennedy, Hilal e Momeni [?] investigaram especificamente o papel do FL na segurança financeira, propondo uma taxonomia baseada em níveis de exposição regulatória. Os autores classificam a detecção de fraude em abertura de conta como uma tarefa de “alta exposição regulatória”, argumentando que o uso de FL não é apenas uma vantagem técnica, mas uma necessidade de *compliance*. Notavelmente, o estudo documenta que a colaboração Cross-Silo entre bancos via FL pode reduzir taxas de falsos positivos em até 15% comparado com modelos isolados.

Claus, John e John [?] focaram especificamente em redes financeiras colaborativas, argumentando que fraudes complexas que “saltam” de banco para banco só podem ser efetivamente detectadas através de modelos treinados colaborativamente. Os autores propõem arquiteturas onde múltiplas instituições contribuem para um modelo global capaz de identificar padrões que seriam invisíveis para modelos treinados em silos isolados.

Lynch, Abdelmoniem e Gill [?] forneceram uma comparação direta entre abordagens centralizadas e descentralizadas para detecção de fraude. Os autores observaram que, embora a abordagem centralizada tenha tipicamente uma ligeira vantagem em acurácia absoluta devido à visibilidade total dos dados, a abordagem federada oferece robustez superior contra ataques de inversão de modelo e cumpre requisitos regulatórios que a centralizada não consegue atender. Esta análise é particularmente relevante para quantificar o “custo da privacidade” em sistemas federados.

2.4 Heterogeneidade de Dados e Fairness

Um dos desafios fundamentais em sistemas de Aprendizado Federado é a natureza não-IID (*Non-Independent and Identically Distributed*) dos dados distribuídos entre participantes. Zhu et al. [?] apresentaram um survey abrangente sobre este problema, documentando como a heterogeneidade estatística entre clientes pode degradar significativamente a performance de modelos globais. Os autores categorizam tipos de viés de dados,

incluindo desbalanceamento de rótulos (*label skew*), desbalanceamento de características (*feature skew*) e desbalanceamento de quantidade (*quantity skew*), todos relevantes para cenários de fraude bancária onde diferentes instituições possuem perfis de clientes distintos.

Li et al. [?] propuseram o FedProx como resposta à heterogeneidade, introduzindo um termo de regularização que limita a divergência entre modelos locais e o modelo global. Karimireddy et al. [?] desenvolveram o SCAFFOLD, que utiliza variáveis de controle para corrigir a deriva de clientes causada por dados heterogêneos. Ambas as abordagens representam alternativas ao FedAvg original que podem ser consideradas em trabalhos futuros para mitigar efeitos de dados Non-IID.

A questão de justiça algorítmica (*fairness*) em ambientes federados foi investigada por Huang et al. [?], que estudaram especificamente o trade-off entre fairness e acurácia em Aprendizado Federado Horizontal. Este trabalho é diretamente relevante para o dataset BAF, que é explicitamente projetado para avaliação de viés demográfico. Os autores demonstram que, ao distribuir dados entre clientes, existe um compromisso complexo entre manter a precisão global e garantir que o modelo não discrimina grupos específicos de usuários.

Zhang, Kou e Wang [?] propuseram o FairFL, uma abordagem para reduzir viés demográfico em modelos federados preservando privacidade. Os autores argumentam que a mitigação de viés em ambientes federados apresenta desafios únicos, pois atributos demográficos sensíveis estão distribuídos entre participantes e não podem ser centralizados para correção. Hardt, Price e Srebro [?] formalizaram conceitos fundamentais de justiça algorítmica como *Equality of Opportunity* (igualdade nas taxas de verdadeiros positivos entre grupos). No contexto deste trabalho, a métrica de *fairness* adotada segue a definição do artigo do BAF [?], que utiliza o *FPR Ratio* — a razão entre as taxas de falso positivo de diferentes grupos demográficos — para avaliar se o modelo penaliza desproporcionalmente determinados grupos com falsos alertas de fraude.

2.5 Privacidade e Segurança

A preservação de privacidade em sistemas FL vai além da simples não-centralização de dados. Bonawitz et al. [?] propuseram protocolos de Agregação Segura (*Secure Aggregation*) que garantem que o servidor central não tenha acesso às atualizações individuais dos clientes, apenas ao resultado agregado. Esta técnica é fundamental para sistemas federados em ambientes financeiros regulados.

Geyer, Klein e Nabi [?] investigaram a aplicação de Privacidade Diferencial (*Differential Privacy*) em FL, propondo mecanismos que adicionam ruído estatístico às atualizações para garantir privacidade formal. Wei et al. [?] formalizaram o trade-off entre

privacidade e utilidade, demonstrando como a performance degrada à medida que garantias de privacidade aumentam. Esta análise é relevante para contextualizar resultados onde modelos federados apresentam performance inferior aos centralizados.

A robustez contra ataques adversariais foi abordada por Miao et al. [?], que propuseram mecanismos para proteção contra ataques bizantinos onde participantes mal-intencionados tentam corromper o modelo global. Esta preocupação é particularmente relevante em cenários bancários, onde a integridade do modelo de detecção de fraude é crítica para a segurança do sistema financeiro.

2.6 Análise Comparativa e Posicionamento do Trabalho

A Tabela 1 apresenta uma análise comparativa dos principais trabalhos relacionados, destacando suas contribuições e limitações em relação ao presente estudo.

Trabalho	Contribuição Principal	Limitação / Lacuna
SecureBoost [?]	Framework lossless para XGBoost federado	Não avalia LightGBM e CatBoost
FedTree [?]	Sistema otimizado para árvores federadas	Foco em eficiência, não em fraude
Kennedy et al. [?]	Taxonomia de FL em finanças	Análise teórica, sem experimentos comparativos
Farrukh et al. [?]	Revisão ML vs FL para fraude	Não compara algoritmos de boosting
BAF Dataset [?]	Dataset benchmark para fraude	Avalia apenas LightGBM centralizado
Huang et al. [?]	Trade-off fairness/accuracy em HFL	Não aplica a detecção de fraude
Este trabalho	Comparação XGBoost/-LightGBM/CatBoost federados no BAF com análise de fairness	—

Tabela 1: Análise comparativa dos trabalhos relacionados

O presente trabalho diferencia-se da literatura existente em três aspectos principais. Primeiro, realiza uma comparação sistemática de três algoritmos de Gradient Boosting (XGBoost, LightGBM e CatBoost) em ambiente federado, preenchendo uma lacuna identificada na literatura onde estudos focam em algoritmos individuais. Segundo, utiliza o dataset BAF — o benchmark mais recente e realista para detecção de fraude em abertura de contas — estendendo a avaliação original do artigo [?] para o contexto federado. Terceiro, avalia o desempenho dos modelos em cenário federado com particionamento IID balanceado, estabelecendo uma *baseline* sólida para futuras investigações com dados heterogêneos (Non-IID), e contribui com análises de *fairness* em modelos de fraude federados.

De acordo com Liu et al. [?], uma das direções abertas no campo é a personalização e eficiência de modelos federados. O presente trabalho ataca diretamente estas lacunas ao demonstrar que modelos de árvore são mais leves que redes neurais profundas para FL, e ao avaliar estratégias de agregação (Bagging e Cíclica) que exploram as características específicas de ensembles baseados em boosting.

3 FUNDAMENTOS TEÓRICOS

Este capítulo apresenta os conceitos fundamentais que sustentam o desenvolvimento deste trabalho. Inicialmente, são discutidos os princípios do Aprendizado Federado, incluindo suas características essenciais, arquitetura cliente-servidor, formalização matemática do processo de otimização distribuída e as diferentes estratégias de agregação que determinam como os modelos locais são combinados para formar o modelo global. Em seguida, são detalhados os modelos baseados em árvore de decisão utilizados neste estudo: XGBoost, LightGBM e CatBoost, explorando suas formulações matemáticas, algoritmos de otimização, estratégias de regularização e características que os tornam adequados para ambientes de Aprendizado Federado e, em particular, para detecção de fraudes bancárias. Por fim, é apresentado o framework Flower, ferramenta central para a implementação de sistemas de Aprendizado Federado, descrevendo sua arquitetura modular, protocolo de comunicação baseado em gRPC, interfaces de programação para clientes e servidores, e os modos de simulação e deployment que facilitam tanto a pesquisa acadêmica quanto aplicações em produção.

3.1 Aprendizado Federado

3.1.1 Definição e Características

O Aprendizado Federado (*Federated Learning* - FL) é um paradigma de aprendizado de máquina distribuído que permite o treinamento de modelos em dados descentralizados, sem a necessidade de centralizar ou compartilhar os dados brutos [?]. Neste modelo, múltiplos dispositivos ou servidores (denominados clientes) colaboram para treinar um modelo global, mantendo seus dados localmente armazenados e privados [?]. Esta abordagem representa uma mudança fundamental em relação ao paradigma tradicional de aprendizado centralizado, onde todos os dados são coletados e armazenados em um único servidor para treinamento [?]. O Aprendizado Federado surgiu como resposta crescente a preocupações com privacidade de dados, regulamentações como GDPR (*General Data Protection Regulation*) [?] e LGPD (Lei Geral de Proteção de Dados), e limitações práticas de transferência de grandes volumes de dados através da rede [?]. Além de preservar a privacidade, o FL permite o aproveitamento de dados distribuídos em dispositivos edge, como smartphones, veículos autônomos, dispositivos IoT e sistemas médicos, sem violar restrições de privacidade ou incorrer em custos proibitivos de comunicação [?, ?].

O fluxo de trabalho do Aprendizado Federado segue 5 etapas ilustradas nas Figuras 1 a 5:



Figura 1: Inicialização do modelo global no servidor central. Fonte: [?]

Na Figura 1, o servidor central inicializa um modelo global que serve como ponto de partida para o treinamento federado. Esta inicialização pode ser feita com parâmetros aleatórios, seguindo distribuições estatísticas apropriadas (como Xavier ou He initialization para redes neurais), ou pode utilizar modelos pré-treinados (transfer learning) quando disponíveis. A qualidade da inicialização pode impactar significativamente a velocidade de convergência e o desempenho final do modelo federado.



Figura 2: Distribuição do modelo global para os clientes. Fonte: [?]

Na Figura 2, o modelo global é transmitido do servidor para os clientes selecionados através da serialização dos parâmetros do modelo. Este processo de broadcast envolve a conversão dos parâmetros do modelo em um formato adequado para transmissão pela rede (como arrays de bytes ou Protocol Buffers), seguido pela distribuição através de protocolos de comunicação eficientes como gRPC. A serialização deve ser eficiente para minimizar o overhead de comunicação, especialmente em cenários com largura de banda limitada ou grande número de clientes participantes.



Figura 3: Treinamento local nos clientes com dados privados. Fonte: [?]

Na Figura 3, cada cliente treina o modelo utilizando seus dados locais privados, sem que os dados saiam do dispositivo cliente. Durante esta fase, cada cliente executa múltiplas épocas de treinamento local usando algoritmos de otimização como SGD (Stochastic Gradient Descent), Adam ou outros otimizadores apropriados para o tipo de modelo. O número de épocas locais e o tamanho do batch são hiperparâmetros importantes que afetam tanto a qualidade do modelo atualizado quanto o custo computacional no cliente. Esta etapa é crucial para preservar a privacidade dos dados, pois nenhuma informação bruta é compartilhada com o servidor ou outros clientes.



Figura 4: Envio das atualizações locais para o servidor. Fonte: [?]

Na Figura 4, após o treinamento local, cada cliente envia suas atualizações de modelo (parâmetros treinados) de volta ao servidor central. As atualizações podem ser enviadas como parâmetros completos do modelo ou, em algumas implementações, como gradientes ou deltas em relação ao modelo inicial. Esta fase de upload representa um dos principais gargalos de comunicação em sistemas FL, especialmente para modelos grandes ou em ambientes com conectividade limitada. Técnicas de compressão, quantização e agregação parcial podem ser empregadas para reduzir o volume de dados transmitidos durante esta etapa.



Figura 5: Agregação das atualizações no servidor. Fonte: [?]

Na Figura 5, o servidor agrega as atualizações recebidas de todos os clientes para criar um novo modelo global melhorado. O ciclo se repete por múltiplas rodadas até a convergência. O processo de agregação é determinado pela estratégia escolhida (como FedAvg, FedProx, ou estratégias específicas para modelos baseados em árvore como Bagging e Cyclic), e pode envolver ponderação das atualizações baseada no número de amostras de cada cliente, qualidade dos dados, ou outros critérios. A convergência é tipicamente avaliada através de métricas como acurácia em um conjunto de validação centralizado, estabilização da função de perda, ou após um número pré-determinado de rodadas de comunicação. Este processo iterativo continua até que um critério de parada seja satisfeito, resultando no modelo global final que incorpora conhecimento de todos os clientes participantes sem ter acesso direto aos seus dados privados.

O processo básico do Aprendizado Federado pode ser descrito formalmente da seguinte forma. Considere N clientes, cada um com seu próprio conjunto de dados local $\mathcal{D}_i$, onde $i = 1, 2, \dots, N$. O objetivo é aprender um modelo global w que minimize a função de perda agregada:

$$\min_w F(w) = \sum_{i=1}^N \frac{n_i}{n} F_i(w) \quad (1)$$

onde $F_i(w)$ é a função de perda local do cliente i , $n_i = |\mathcal{D}_i|$ é o número de amostras do cliente i , e $n = \sum_{i=1}^N n_i$ é o total de amostras em todos os clientes. A função de perda local é definida como:

$$F_i(w) = \frac{1}{n_i} \sum_{(x,y) \in \mathcal{D}_i} \ell(w; x, y) \quad (2)$$

onde $\ell(w; x, y)$ é a função de perda para uma amostra individual (x, y) com parâmetros do modelo w .

As 5 principais características do Aprendizado Federado que o diferenciam do aprendizado de máquina tradicional centralizado são:

Privacidade e Segurança: Os dados nunca saem dos dispositivos locais dos clientes. Apenas atualizações de modelo (gradientes ou parâmetros) são compartilhadas com o servidor central, preservando a privacidade dos dados sensíveis. Esta característica é especialmente importante em domínios como saúde, finanças e dispositivos móveis.

Comunicação Limitada: A troca de dados entre clientes e servidor é reduzida ao compartilhamento de parâmetros do modelo, ao invés de conjuntos de dados completos. Isso minimiza o overhead de comunicação, crucial em ambientes com largura de banda limitada.

Dados Não-IID: Os dados distribuídos entre os clientes frequentemente não são independentes e identicamente distribuídos (Non-IID) [?]. Cada cliente pode ter uma distribuição de dados significativamente diferente dos demais, refletindo características locais específicas. Segundo Li et al. [?], esta heterogeneidade estatística representa um dos principais desafios do FL, podendo causar divergência nos modelos locais e degradação da performance global.

Desbalanceamento de Dados: O número de amostras pode variar drasticamente entre clientes. Alguns clientes podem ter milhares de amostras enquanto outros possuem apenas algumas dezenas. Este desbalanceamento requer estratégias de agregação ponderada para evitar que clientes com mais dados dominem o modelo global [?].

Heterogeneidade de Dispositivos: Os clientes podem ter capacidades computacionais, de armazenamento e de rede muito diferentes [?]. Esta heterogeneidade exige estratégias adaptativas para garantir a convergência do modelo, como as propostas por Karimireddy et al. [?] com o algoritmo SCAFFOLD.

O ciclo de treinamento federado típico segue 6 etapas:

1. **Inicialização:** O servidor central inicializa o modelo global $w^{(0)}$ com parâmetros aleatórios ou pré-treinados.

2. **Seleção de Clientes:** Em cada rodada t , o servidor seleciona um subconjunto $\mathcal{S}_t$ de clientes disponíveis para participar do treinamento.
3. **Broadcast:** O servidor envia os parâmetros atuais do modelo global $w^{(t)}$ para os clientes selecionados.
4. **Treinamento Local:** Cada cliente $i \in \mathcal{S}_t$ realiza treinamento local usando seus dados $\mathcal{D}_i$, produzindo parâmetros atualizados $w_i^{(t+1)}$.
5. **Agregação:** O servidor coleta os modelos locais dos clientes e os agrega para produzir o novo modelo global $w^{(t+1)}$.
6. **Convergência:** O processo se repete até que um critério de convergência seja atingido (número de rodadas, precisão mínima, etc.).

3.1.2 Estratégias de Agregação

As estratégias de agregação são fundamentais no Aprendizado Federado, pois determinam como os modelos locais treinados pelos clientes são combinados para formar o modelo global [?]. A escolha da estratégia de agregação impacta diretamente na convergência, precisão e robustez do sistema federado, influenciando aspectos como velocidade de convergência, qualidade do modelo final, tolerância a dados Non-IID, consumo de recursos de comunicação e capacidade de lidar com clientes heterogêneos ou falhas de comunicação [?]. Diferentes estratégias são mais adequadas para diferentes cenários de aplicação, tipos de modelos e características dos dados distribuídos. Neste trabalho, são exploradas 2 estratégias principais de agregação especialmente relevantes para modelos baseados em árvore: Bagging e Cíclica (Cyclic), conforme implementadas no framework Flower para modelos de *Gradient Boosting* [?].

Estratégia Bagging

Na estratégia de Bagging, múltiplos clientes treinam modelos independentes em paralelo usando seus dados locais. Após o treinamento, os modelos são agregados no servidor central. Para modelos baseados em árvore de decisão, a agregação por Bagging é particularmente eficaz, pois permite combinar os conjuntos (*ensembles*) de árvores de diferentes clientes — isto é, os grupos de múltiplas árvores de decisão que cada modelo local produz durante o treinamento — explorando a diversidade dos dados locais para criar um modelo global mais robusto e generalizável. Esta estratégia é análoga ao tradicional *Bootstrap Aggregating* (Bagging) proposto por Breiman, mas adaptada para o contexto federado onde os dados já estão naturalmente particionados entre clientes. Considere que cada cliente i treina um modelo M_i com K árvores. A agregação pode ser realizada de 2 formas distintas, cada uma com suas vantagens e características:

Agregação de Árvores: As árvores individuais de todos os modelos locais são combinadas em um único ensemble:

$$M_{global} = \bigcup_{i \in \mathcal{S}} \text{Árvores}(M_i) \quad (3)$$

Agregação de Predições: Cada modelo local realiza predições independentes, e a predição final é obtida pela média (regressão) ou votação (classificação):

$$\hat{y}_{ensemble}(x) = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} M_i(x) \quad (4)$$

Estratégia Cíclica (Cyclic)

Na estratégia Cíclica, diferentemente do treinamento paralelo do Bagging, os clientes treinam sequencialmente. Em cada rodada t , apenas um cliente i_t é selecionado para treinar, e o modelo é passado de um cliente para o próximo em uma ordem determinada:

$$w^{(t+1)} = w_i^{(t+1)}, \quad \text{onde } i = tN \quad (5)$$

O processo pode ser descrito em 4 etapas:

1. Cliente i_1 recebe o modelo inicial $w^{(0)}$ e treina localmente, produzindo $w_1^{(1)}$
2. Cliente i_2 recebe $w_1^{(1)}$, treina e produz $w_2^{(2)}$
3. O processo continua até todos os clientes participarem
4. O ciclo pode se repetir por múltiplas rodadas

Esta estratégia é vantajosa quando há limitações de recursos (memória, largura de banda) ou quando se deseja um processo de convergência mais gradual e controlado. Para modelos baseados em árvore, a estratégia cíclica permite que cada cliente adicione árvores incrementalmente ao modelo existente.

3.2 Modelos Baseados em Árvore de Decisão

Os modelos baseados em árvore de decisão são algoritmos de aprendizado de máquina que utilizam estruturas em forma de árvore para realizar predições. Estes modelos são particularmente eficazes para dados tabulares e têm sido amplamente utilizados em competições de ciência de dados e sistemas de produção no setor financeiro devido à sua precisão, interpretabilidade e eficiência computacional [?, ?, ?]. De acordo com Liu et

al. [?], modelos de *Gradient Boosting* baseados em árvore representam o estado da arte para dados tabulares, oferecendo vantagens significativas sobre redes neurais profundas em termos de eficiência de treinamento e comunicação, aspectos críticos para ambientes de Aprendizado Federado.

Um modelo baseado em árvore de decisão faz previsões dividindo recursivamente o espaço de características em regiões. Cada divisão é determinada por uma condição sobre uma característica específica. Para uma tarefa de regressão, a predição final para uma amostra x é dada por:

$$\hat{y}(x) = \sum_{j=1}^J w_j 1(x \in R_j) \quad (6)$$

onde J é o número de folhas na árvore, w_j é o valor de predição associado à folha j , R_j é a região definida pela folha j , e $1(\cdot)$ é a função indicadora.

Os métodos de ensemble combinam múltiplas árvores de decisão para melhorar a precisão e generalização. Os 3 algoritmos de *Gradient Boosting* considerados neste trabalho (XGBoost, LightGBM e CatBoost) são baseados no princípio de boosting, onde árvores são adicionadas sequencialmente para corrigir os erros das árvores anteriores.

3.2.1 XGBoost

XGBoost (*eXtreme Gradient Boosting*) [?] é um algoritmo de *Gradient Boosting* otimizado que se tornou uma das técnicas mais populares em aprendizado de máquina para dados tabulares. O algoritmo constrói um ensemble de árvores de decisão de forma aditiva, onde cada nova árvore é treinada para corrigir os erros residuais das árvores anteriores. A adaptação do XGBoost para ambientes federados foi formalizada por Cheng et al. [?] com o SecureBoost, demonstrando que é possível manter precisão comparável ao treinamento centralizado enquanto preserva-se a privacidade dos dados.

O modelo XGBoost para uma amostra x_i após K iterações é dado por:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i) \quad (7)$$

onde f_k representa a k -ésima árvore de decisão. O objetivo do treinamento é minimizar a seguinte função de perda regularizada:

$$\mathcal{L}(\phi) = \sum_{i=1}^n \ell(\hat{y}_i, y_i) + \sum_{k=1}^K \Omega(f_k) \quad (8)$$

onde ℓ é uma função de perda diferenciável (ex: erro quadrático médio, log-loss), e $\Omega(f_k)$

é um termo de regularização definido por:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (9)$$

onde T é o número de folhas, w_j é o peso da folha j , γ penaliza a complexidade do modelo (número de folhas), e λ controla a regularização L2 nos pesos das folhas.

Utilizando uma expansão de Taylor de segunda ordem e otimização aditiva, o XGBoost obtém o peso ótimo para cada folha j da árvore:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \quad (10)$$

onde I_j é o conjunto de amostras atribuídas à folha j , g_i é o gradiente de primeira ordem e h_i é a Hessiana da função de perda. O ganho de uma divisão que separa o conjunto I em I_L (esquerda) e I_R (direita) é dado por:

$$Ganho = \frac{1}{2} \left[\frac{\left(\sum_{i \in I_L} g_i \right)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{\left(\sum_{i \in I_R} g_i \right)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{\left(\sum_{i \in I} g_i \right)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma \quad (11)$$

Além da formulação matemática, o XGBoost inclui otimizações de eficiência como *column block* para armazenamento paralelo, suporte a regularização L1 e L2 para prevenir *overfitting*, treinamento nativo em GPU e tratamento automático de valores ausentes.

3.2.2 LightGBM

LightGBM (*Light Gradient Boosting Machine*) [?] é um framework de *Gradient Boosting* desenvolvido pela Microsoft que utiliza algoritmos baseados em árvores de decisão. A principal inovação do LightGBM é o uso de estratégias de crescimento de árvore mais eficientes, resultando em treinamento mais rápido e menor uso de memória. Segundo o estudo de Li, Wen e He [?], é possível manter a eficiência computacional característica do LightGBM em ambientes distribuídos, tornando-o particularmente adequado para sistemas de Aprendizado Federado onde a eficiência de comunicação é crítica [?].

Enquanto a maioria dos algoritmos de *Gradient Boosting* cresce árvores *level-wise* (nível por nível), dividindo todos os nós no mesmo nível, o LightGBM utiliza uma estratégia *leaf-wise* (folha por folha), que divide sempre a folha que maximiza a redução de perda, mesmo que isso resulte em árvores desbalanceadas. Esta abordagem geralmente converge mais rapidamente.

O LightGBM introduz 2 técnicas principais para melhorar a eficiência:

Gradient-based One-Side Sampling (GOSS): Reduz o número de amostras usadas para estimar o ganho de informação, mantendo aquelas com gradientes grandes (erros grandes) e amostrando aleatoriamente aquelas com gradientes pequenos. Ao manter as $\text{top-}a \times 100\%$ amostras com maiores gradientes e amostrar $b \times 100\%$ das restantes, o GOSS preserva a qualidade da estimativa de ganho com custo computacional significativamente menor.

Exclusive Feature Bundling (EFB): Reduz a dimensionalidade agrupando características mutuamente exclusivas (features que raramente assumem valores não-zero simultaneamente), aproximando o problema como coloração de grafos.

Além dessas técnicas, o LightGBM utiliza histogramas para discretizar características contínuas em bins discretos, reduzindo a complexidade de encontrar a melhor divisão de $O(n \times d)$ para $O(b \times d)$, onde $b \ll n$. As vantagens do LightGBM incluem velocidade de treinamento superior, menor uso de memória e suporte nativo a características categóricas sem necessidade de *one-hot encoding*.

3.2.3 CatBoost

CatBoost (*Categorical Boosting*) [?] é um algoritmo de *Gradient Boosting* desenvolvido pela Yandex que se diferencia por seu tratamento superior de características categóricas e por mecanismos que reduzem *overfitting*. O nome reflete sua principal inovação: o tratamento eficiente de variáveis categóricas sem a necessidade de pré-processamento extensivo. O CatBoost não foi avaliado no artigo original do dataset BAF [?], representando uma contribuição deste trabalho ao estender a análise para este algoritmo tanto em cenário centralizado quanto federado.

Ordered Boosting: Uma limitação dos algoritmos tradicionais de *Gradient Boosting* é o chamado *prediction shift*: o modelo usado para calcular gradientes é treinado nos mesmos dados, levando a um viés. CatBoost resolve isso com *Ordered Boosting*, onde para cada amostra x_i , o gradiente é calculado usando um modelo M_i treinado apenas nas amostras que precedem x_i em uma permutação aleatória, eliminando o viés de treinamento.

Target Statistics para Características Categóricas: CatBoost codifica características categóricas calculando estatísticas do target baseadas nos valores categóricos. Para uma característica categórica c com valor v , a codificação é:

$$TS(c = v) = \frac{\sum_{i:c_i=v, i < j} y_i + a \cdot P}{\sum_{i:c_i=v, i < j} 1 + a} \quad (12)$$

onde a soma considera apenas amostras que precedem a amostra atual j na permutação (evitando *data leakage* — vazamento de informações do conjunto de teste para o treinamento), P é uma estimativa *prior* do target (geralmente a média global) e $a > 0$ é um parâmetro de suavização.

Oblivious Trees: CatBoost utiliza árvores simétricas (*oblivious trees*) como modelos base, onde a mesma condição de divisão é aplicada em todos os nós do mesmo nível. Uma árvore de profundidade d possui 2^d folhas e sua estrutura balanceada atua como forma de regularização, além de permitir previsões computacionalmente eficientes. O CatBoost também implementa múltiplas formas de regularização, incluindo penalidade de complexidade, subamostragem com *Bayesian bootstrap* e adição de ruído aleatório às pontuações de divisão (*random strength*).

3.3 Framework Flower

Flower (*Federated Learning over the Wire*) [?] é um framework open-source para Aprendizado Federado que facilita a implementação e experimentação de sistemas FL. Desenvolvido para ser agnóstico em relação ao framework de machine learning subjacente (PyTorch, TensorFlow, scikit-learn, XGBoost, etc.), Flower oferece uma arquitetura flexível e escalável para pesquisa e aplicações em produção. Na perspectiva de Liu et al. [?], frameworks como Flower representam a infraestrutura fundamental para pesquisa em FL, embora alternativas industriais como o FATE [?] da WeBank ofereçam implementações mais robustas de SecureBoost para cenários de produção. O design do Flower é fundamentado em três princípios principais: flexibilidade, permitindo suporte a diferentes frameworks de ML, estratégias de agregação e topologias de rede para experimentação com diversos algoritmos e configurações; extensibilidade, através de uma arquitetura modular que permite fácil customização de componentes individuais (clientes, servidor, estratégias) sem modificar o núcleo do framework; e simplicidade, oferecendo uma API minimalista e intuitiva que reduz a complexidade de implementação de sistemas FL, tornando-os acessíveis a pesquisadores e desenvolvedores. A Figura 6 ilustra a arquitetura do framework Flower, demonstrando sua capacidade de integrar diversos tipos de clientes e algoritmos de aprendizado de máquina.

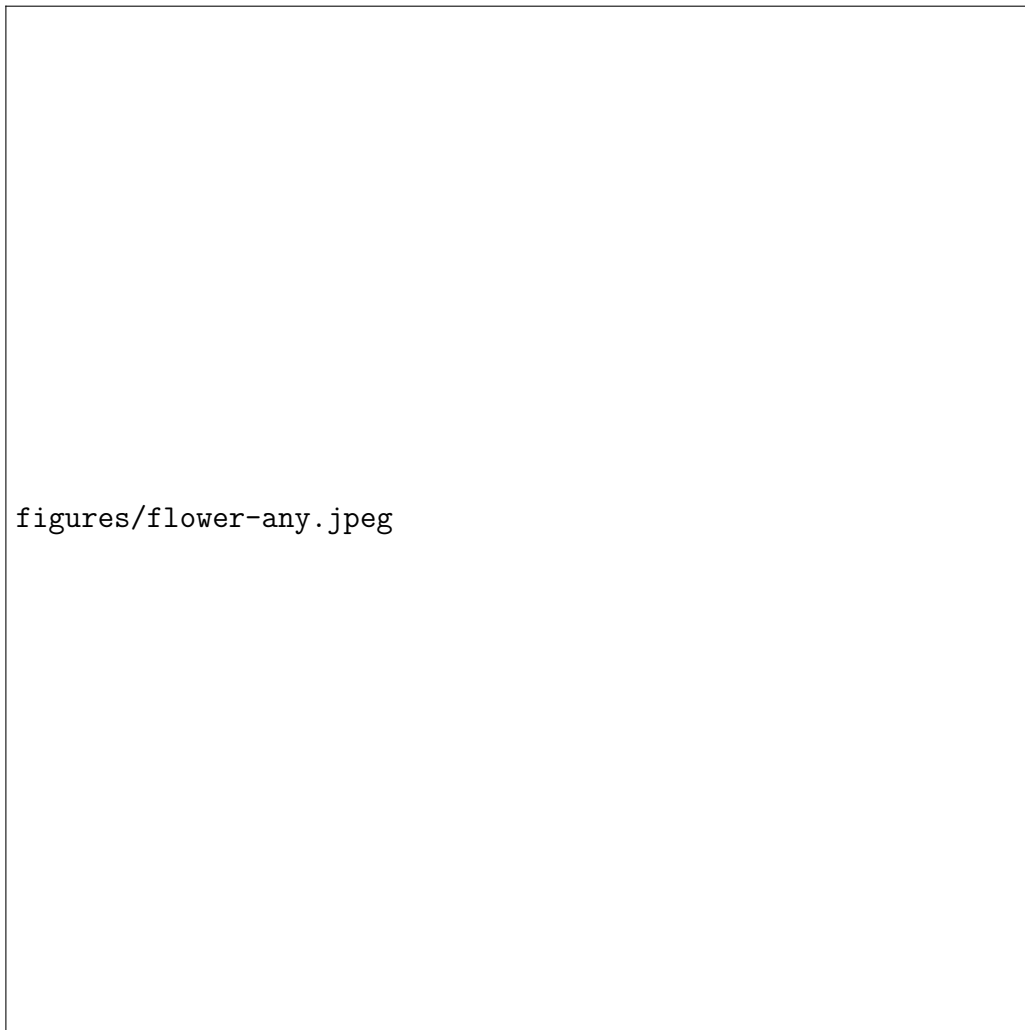


Figura 6: Arquitetura do Flower: framework agnóstico que suporta diversos tipos de clientes e algoritmos de ML. Fonte: [?]

3.3.1 Arquitetura do Flower

A arquitetura do Flower segue um modelo cliente-servidor com comunicação baseada em gRPC (*Google Remote Procedure Call*), permitindo comunicação eficiente entre clientes e servidor através de diferentes redes e plataformas. O framework é estruturado em torno de 3 componentes fundamentais — Cliente, Servidor e Estratégia — que interagem de forma coordenada para viabilizar o treinamento federado.

O componente *Client* (Cliente) representa um participante do treinamento federado que possui dados locais. Cada cliente implementa a interface `Client` do Flower, que define 2 métodos principais: `fit(parameters, config)`, responsável por receber os parâmetros do modelo global, realizar o treinamento local utilizando os dados privados do cliente e retornar os parâmetros atualizados junto com métricas de treinamento; e `evaluate(parameters, config)`, que avalia o modelo global nos dados locais de validação e retorna métricas de desempenho. Formalmente, um cliente i executa as seguintes

computações durante uma rodada de treinamento:

$$w_i^{(t+1)} = \text{fit}(w^{(t)}, \mathcal{D}_i, \text{config})$$

$$\text{métricas}_i = \text{evaluate}(w^{(t+1)}, \mathcal{D}_i^{\text{val}})$$

O componente *Server* (Servidor) coordena todo o processo de treinamento federado, sendo responsável por gerenciar os clientes participantes, distribuir os parâmetros do modelo global e agregar as atualizações recebidas. O servidor mantém o modelo global $w^{(t)}$ e orquestra as rodadas de treinamento conforme descrito no Algoritmo 1. Em cada rodada, o servidor seleciona um subconjunto de clientes disponíveis, envia os parâmetros atuais, aguarda as atualizações locais e, por fim, agrega os resultados para produzir o novo modelo global.

[H] [1] Inicializar modelo global $w^{(0)}$ rodada $t = 1$ até T Selecionar clientes $\mathcal{S}_t \subseteq \mathcal{C}$ usando `configure_fit()` Enviar $w^{(t)}$ e configuração para clientes em $\mathcal{S}_t$ Receber atualizações $\{w_i^{(t+1)}\}_{i \in \mathcal{S}_t}$ dos clientes $w^{(t+1)} \leftarrow \text{aggregate_fit}()(\{w_i^{(t+1)}\}_{i \in \mathcal{S}_t})$ (Opcional) Avaliar $w^{(t+1)}$ usando `evaluate()` **Retornar** $w^{(T)}$

O componente *Strategy* (Estratégia) define a lógica de agregação dos modelos locais e a política de seleção de clientes. Uma estratégia no Flower implementa funções como `configure_fit()`, que seleciona os clientes participantes e prepara as configurações de cada rodada; `aggregate_fit()`, que agrega os resultados de treinamento dos clientes para produzir o modelo global atualizado; e as funções correspondentes para avaliação distribuída (`configure_evaluate()` e `aggregate_evaluate()`). Esta separação entre servidor e estratégia permite que diferentes algoritmos de agregação (como FedAvg, Bagging ou Cyclic) sejam utilizados sem modificar a lógica do servidor ou dos clientes.

A comunicação entre os componentes é realizada através de *Protocol Buffers* e gRPC, que definem mensagens estruturadas para a troca eficiente de parâmetros e metadados. O protocolo encapsula os parâmetros do modelo como *arrays* de bytes na estrutura `Parameters`, enquanto as instruções e resultados de treinamento são transmitidos através de pares de mensagens complementares: `FitIns/FitRes` para o treinamento e `EvaluateIns/EvaluateRes` para a avaliação. Este design permite a serialização e deserialização eficiente dos modelos, minimizando o *overhead* de comunicação entre clientes e servidor.

O Flower oferece 2 modos de operação que atendem a diferentes necessidades. O modo de simulação (*Simulation Mode*) permite simular múltiplos clientes em uma única máquina utilizando o Ray para processamento distribuído, sendo especialmente útil para pesquisa e experimentação rápida, como é o caso deste trabalho. O modo de *deployment*, por sua vez, executa clientes e servidor em processos ou máquinas separadas, comunicando-se através da rede, sendo adequado para ambientes de produção e cenários *cross-device* reais. Em ambos os modos, o ciclo de vida de uma rodada de treinamento

segue o mesmo fluxo: o servidor seleciona os clientes participantes, distribui os parâmetros globais, aguarda o treinamento local e o envio das atualizações, agrega os resultados recebidos para atualizar o modelo global e, opcionalmente, realiza uma avaliação centralizada ou distribuída do novo modelo antes de iniciar a próxima rodada.

4 METODOLOGIA

4.1 Visão Geral da Solução Proposta

Este trabalho propõe uma solução de Aprendizado Federado para detecção de fraudes bancárias utilizando modelos baseados em árvore de decisão. A abordagem consiste em treinar modelos XGBoost [?], LightGBM [?] e CatBoost [?] de forma distribuída, onde cada cliente (representando uma instituição financeira ou filial) mantém seus dados localmente e colabora no treinamento de um modelo global através do framework Flower [?]. Esta arquitetura permite que múltiplas instituições se beneficiem de um modelo mais robusto e generalizável, treinado em dados diversos, sem comprometer a privacidade dos dados sensíveis dos clientes bancários [?, ?].

A escolha por modelos de *Gradient Boosting* em detrimento de redes neurais profundas é fundamentada em três argumentos principais documentados na literatura. Primeiro, conforme demonstrado por Cheng et al. [?] com o SecureBoost, modelos baseados em árvore podem atingir precisão comparável ao treinamento centralizado (aproximadamente 90% em benchmarks de fraude) enquanto preservam a privacidade. Segundo, Li et al. [?] mostraram que sistemas de árvore otimizados são ordens de magnitude mais eficientes que implementações baseadas em redes neurais, aspecto crítico para detecção de fraude em tempo real. Terceiro, conforme argumentado por Liu et al. [?], modelos de árvore oferecem resistência natural superior a ataques de vazamento de informação (*Deep Leakage*) [?], uma vez que suas atualizações são baseadas em histogramas e divisões, dificultando a reconstrução dos dados originais.

O sistema avalia duas estratégias de agregação distintas — Bagging e Cíclica — para determinar qual abordagem produz os melhores resultados no domínio de detecção de fraudes em abertura de contas bancárias, utilizando o dataset BAF [?] como base experimental.

4.2 Arquitetura do Sistema

A arquitetura do sistema segue o paradigma cliente-servidor característico do Aprendizado Federado Horizontal (HFL), conforme categorizado por Yang et al. [?]. No HFL, os participantes compartilham o mesmo espaço de atributos mas possuem amostras diferentes — exatamente o cenário de instituições financeiras que coletam os mesmos tipos de dados de clientes distintos.

O servidor central é responsável por coordenar o treinamento federado, executando as seguintes funções: inicializar o modelo global, selecionar clientes para participação em cada rodada, distribuir os parâmetros do modelo, agregar as atualizações recebidas dos

clientes utilizando a estratégia escolhida (Bagging ou Cíclica), e avaliar o modelo global utilizando um conjunto de teste centralizado [?]. Esta arquitetura segue as melhores práticas estabelecidas por Bonawitz et al. [?] para sistemas FL em escala.

Cada cliente representa um participante do sistema federado que possui uma partição local do dataset BAF, simulando diferentes instituições bancárias que colaboram no treinamento de um modelo global. Neste trabalho, os dados são distribuídos entre os clientes seguindo um particionamento IID (*Independent and Identically Distributed*) balanceado, garantindo que cada cliente receba uma amostra representativa e estatisticamente similar do dataset. A comunicação entre clientes e servidor é mediada pelo Flower, que abstrai os detalhes de serialização, transmissão e sincronização através do protocolo gRPC [?].

4.3 Particionamento dos Dados entre Clientes

A partição do dataset BAF entre os clientes federados segue uma estratégia IID (*Independent and Identically Distributed*) balanceada. Nesta abordagem, o conjunto de treinamento é dividido de forma uniforme e aleatória entre os clientes, de modo que cada partição preserve a mesma distribuição estatística do dataset original — incluindo a proporção de classes (fraude vs. legítimo) e a distribuição dos atributos.

A escolha pelo particionamento IID balanceado é motivada pelo objetivo de isolar o impacto do treinamento federado em si, sem a influência adicional da heterogeneidade estatística entre clientes. Esta abordagem permite quantificar com maior precisão a diferença de desempenho entre o treinamento federado e o centralizado, estabelecendo uma *baseline* controlada. De acordo com a literatura [?, ?, ?], cenários com dados Non-IID introduzem desafios adicionais como divergência entre modelos locais e degradação da convergência, que poderiam dificultar a análise comparativa entre os algoritmos de *Gradient Boosting* — foco principal deste trabalho. A investigação de cenários Non-IID, onde diferentes clientes possuem perfis de dados heterogêneos simulando instituições bancárias com características distintas [?], é apontada como direção relevante para trabalhos futuros.

4.4 Implementação dos Modelos Federados

A implementação dos modelos federados segue rigorosamente o padrão de cliente-servidor do framework Flower [?], onde cada algoritmo (XGBoost, LightGBM e CatBoost) possui um cliente customizado que implementa a interface abstrata `Client` do Flower. Esta abordagem é consistente com as práticas recomendadas por Li et al. [?] para implementação de *Gradient Boosting* federado.

Todos os algoritmos implementados neste trabalho compartilham uma estrutura comum de dois métodos obrigatórios:

- `fit(ins: FitIns) -> FitRes`: Realiza o treinamento local utilizando os dados do cliente e retorna o modelo atualizado. Este método implementa a otimização local descrita por McMahan et al. [?].
- `evaluate(ins: EvaluateIns) -> EvaluateRes`: Avalia o modelo global com os dados locais do cliente, retornando métricas de desempenho para agregação no servidor.

A serialização dos modelos varia entre os algoritmos devido às diferenças em suas APIs:

- **XGBoost**: Permite serialização direta e eficiente em formato JSON através do método `save_raw()`, alinhado com a abordagem do SecureBoost [?].
- **LightGBM**: Requer serialização baseada em arquivos temporários, conforme limitações da API. A eficiência é mantida pela natureza compacta dos histogramas [?].
- **CatBoost**: Similar ao LightGBM, utiliza arquivos temporários para serialização no formato binário proprietário `.cbm`.

O treinamento incremental é suportado por todos os três algoritmos através do mecanismo de *warm start* — técnica na qual o treinamento local não é iniciado do zero, mas sim a partir de um modelo previamente treinado, que serve como ponto de partida. A partir da segunda rodada de comunicação, o treinamento local continua a partir do modelo global recebido, preservando o conhecimento acumulado nas rodadas anteriores. Esta característica é fundamental para a convergência eficiente em sistemas federados [?].

4.5 Estratégias de Agregação

Para os três algoritmos, foram implementadas estratégias customizadas que herdam da classe **Strategy** do Flower, permitindo controle completo sobre a seleção de clientes, distribuição de modelos e agregação de resultados.

Estratégia Bagging: Múltiplos clientes treinam modelos independentemente em paralelo, e os resultados são agregados por média das probabilidades preditas (*ensemble averaging*). Na primeira rodada, os clientes treinam do zero; nas rodadas seguintes, o servidor seleciona o melhor modelo da rodada anterior (baseado no TPR de validação) e o envia como ponto de partida (*warm start*) para todos os clientes, permitindo refinamento progressivo. Esta abordagem explora a diversidade dos dados locais para criar um modelo global mais robusto, análoga ao Bootstrap Aggregating original mas adaptada para o contexto federado [?].

Estratégia Cíclica: Os clientes treinam sequencialmente em ordem circular, onde cada cliente recebe o modelo do cliente anterior via *warm start* e adiciona árvores incrementalmente. Para controlar o sobreajuste inerente ao treinamento sequencial, é aplicado um *learning rate decay* exponencial com fator $0,85^{r-1}$, onde r é o número da rodada. Este mecanismo reduz progressivamente a taxa de aprendizado nas rodadas posteriores, permitindo ajustes mais finos e estabilizando a convergência [?].

4.6 Métricas de Avaliação

As métricas são calculadas após cada rodada de treinamento, incluindo:

- **Acurácia:** Proporção de predições corretas sobre o total.
- **Precisão:** Proporção de verdadeiros positivos entre todas as predições positivas.
- **Recall (Sensibilidade):** Proporção de fraudes corretamente identificadas.
- **F1-Score:** Média harmônica entre precisão e recall.
- **AUC-ROC:** Área sob a curva ROC, métrica robusta para dados desbalanceados.

Conforme recomendado por Chai et al. [?], a avaliação de sistemas FL não deve focar apenas na utilidade (performance do modelo), mas também na eficiência de comunicação e convergência. Este trabalho reporta o número de rodadas de comunicação necessárias para atingir diferentes níveis de performance, permitindo análise do trade-off entre acurácia e custo de comunicação.

Para avaliação de *fairness*, adotamos a métrica de *FPR Ratio* definida no artigo do dataset BAF [?], que mede a razão entre as taxas de falso positivo (FPR) de diferentes grupos demográficos — no caso, grupos etários definidos pelo atributo protegido *customer_age*. Um *FPR Ratio* próximo a 1,0 indica tratamento equitativo, ou seja, que clientes legítimos de ambos os grupos possuem probabilidade similar de serem incorretamente classificados como fraudulentos. O artigo do BAF avalia dois regimes de limiar de decisão: o *standard*, que utiliza um único limiar global calibrado para 5% de FPR, e o *fair*, que aplica limiares independentes por grupo demográfico, cada um calibrado para 5% de FPR no respectivo grupo, equalizando efetivamente as taxas de falso positivo entre grupos.

4.7 Tecnologias Utilizadas

Componente	Tecnologia
Framework FL	Flower 1.6+ [?]
Modelos	XGBoost [?], LightGBM [?], CatBoost [?]
Linguagem	Python 3.9+
Comunicação	gRPC (Protocol Buffers)
Dataset	Bank Account Fraud (BAF) [?]
Processamento de Dados	pandas, NumPy, scikit-learn
Métricas de Fairness	FPR Ratio (<i>Predictive Equality</i>) [?]

Tabela 2: Tecnologias utilizadas no projeto

4.8 Limitações Metodológicas

É importante reconhecer as limitações desta abordagem metodológica. Conforme documentado por Wei et al. [?], existe um trade-off inerente entre privacidade e utilidade em sistemas federados. Este trabalho não implementa técnicas adicionais de privacidade como Agregação Segura [?] ou Privacidade Diferencial [?], focando na avaliação de performance dos modelos de árvore em si. A implementação de camadas adicionais de privacidade é deixada para trabalhos futuros.

Adicionalmente, o cenário de simulação utiliza particionamento IID balanceado, que não captura a heterogeneidade estatística (Non-IID) presente em cenários reais onde diferentes instituições possuem perfis de clientes distintos [?]. Além disso, a simulação não replica completamente os desafios de sistemas de produção em escala, como latência de rede variável, falhas de comunicação e ataques adversariais [?].

5 DATASET E PREPARAÇÃO DOS DADOS

5.1 O Dataset Bank Account Fraud (BAF)

Para avaliar o desempenho dos algoritmos de Aprendizado Federado neste trabalho, foi utilizado o dataset *Bank Account Fraud* (BAF) [?], publicado no NeurIPS 2022 (*Track on Datasets and Benchmarks*). O BAF constitui o primeiro conjunto de dados publicamente disponível, de larga escala, realista e com preservação de privacidade, voltado especificamente para a detecção de fraudes em abertura de contas bancárias online. O dataset está disponível publicamente no repositório GitHub do Feedzai¹.

5.1.1 Contexto e Domínio de Aplicação

O dataset BAF aborda a detecção de tentativas fraudulentas de abertura de contas bancárias em um grande banco de consumo. Neste cenário, fraudadores tentam se passar por outra pessoa (roubo de identidade) ou criar uma identidade fictícia para obter acesso aos serviços bancários. Após conseguir acesso a uma nova conta, o fraudador rapidamente esgota a linha de crédito associada ou utiliza a conta para receber pagamentos ilícitos. Todos os custos são suportados pelo banco, já que não há como rastrear a verdadeira identidade do fraudador [?].

Este caso de uso é considerado um domínio de alto impacto (*high-stakes domain*) para aplicação de ML. Uma predição positiva (classificada como fraudulenta) leva à rejeição da aplicação de conta bancária do cliente (ação punitiva), enquanto uma predição negativa leva à concessão de acesso a uma nova conta e seu cartão de crédito (ação assistiva). Possuir uma conta bancária é considerado um direito básico na União Europeia, tornando a detecção de fraudes uma aplicação extremamente pertinente do ponto de vista social [?].

5.1.2 Geração dos Dados e Preservação de Privacidade

O dataset BAF foi gerado a partir de um dataset real e anonimizado de detecção de fraudes em abertura de contas bancárias, utilizando técnicas estado da arte de geração de dados tabulares. Especificamente, foram utilizados modelos *Conditional Tabular Generative Adversarial Network* (CTGAN) [?] para gerar dados sintéticos que preservam as propriedades estatísticas dos dados originais enquanto garantem a privacidade dos solicitantes.

O processo de preservação de privacidade envolveu múltiplas etapas [?]:

¹<https://github.com/feedzai/bank-account-fraud>

1. **Seleção de features:** A partir do dataset original, foram treinadas 200 configurações de hiperparâmetros de LightGBM. As features foram selecionadas com base na importância atribuída pelos 5 melhores modelos dentre essas configurações, resultando em 43 features candidatas, que foram então manualmente reduzidas para 30 features finais visando maximizar expressividade e interpretabilidade.
2. **Adição de ruído Laplaciano:** Aplicação de ruído Laplaciano às colunas do dataset original antes do treinamento da GAN, garantindo um orçamento de privacidade diferencial ϵ . O nível de ruído é inversamente proporcional ao orçamento de privacidade.
3. **Categorização de atributos sensíveis:** Informações sobre idade e renda dos solicitantes foram categorizadas (em faixas) antes do treinamento da GAN, impedindo o acesso a dados específicos dos solicitantes.
4. **Filtragem de duplicatas:** Após a geração, foi garantido que nenhuma instância gerada correspondia exatamente a uma instância do dataset original.

5.1.3 Suite de Datasets e Variantes

O BAF é composto por uma suite de 6 datasets: um dataset base e 5 variantes adicionais, cada uma com padrões controlados de viés de dados. Cada variante segue a mesma distribuição subjacente (todas as instâncias são amostradas do mesmo modelo generativo), mas introduz diferentes tipos de viés para permitir testes de estresse de desempenho e *fairness*. As variantes são descritas na Tabela 3.

Dataset	Grupo	Tamanho	Prevalência	Separabilidade
2*Base	Maioria	77%	0,9%	—
	Minoria	23%	1,8%	—
2*Variante I	Maioria	90%	1,1%	—
	Minoria	10%	1,1%	—
2*Variante II	Maioria	50%	0,4%	—
	Minoria	50%	1,9%	—
2*Variante III	Maioria	50%	1,1%	Aumentada
	Minoria	50%	1,1%	Igual
2*Variante IV	Maioria	50%	0,3% (1,5%)	—
	Minoria	50%	1,7% (1,5%)	—
2*Variante V	Maioria	50%	1,1%	Aumentada (Igual)
	Minoria	50%	1,1%	Igual (Igual)

Tabela 3: Resumo das variantes do dataset BAF. Valores entre parênteses referem-se ao conjunto de teste. Adaptado de [?].

Os tipos de viés introduzidos nas variantes baseiam-se em uma taxonomia de viés de dados [?]:

- **Disparidade de tamanho de grupo:** Frequências diferentes dos grupos do atributo protegido (idade) no dataset.
- **Disparidade de prevalência:** A probabilidade da classe (fraude) depende do grupo protegido.
- **Disparidade de separabilidade:** A distribuição conjunta das features e do rótulo varia entre grupos, afetando a dificuldade de classificação por grupo.

Neste trabalho, utilizamos o dataset **Base** do BAF para os experimentos de Aprendizado Federado, por ser a variante que mais fielmente reproduz as características do dataset original sem vieses adicionais controlados.

5.2 Estrutura e Características do Dataset

A Tabela 4 apresenta os metadados do dataset BAF Base utilizado nos experimentos deste trabalho.

Característica	Valor
Nome	Bank Account Fraud (BAF) — Base
Origem	Dados sintéticos gerados por CTGAN a partir de dados reais anonimizados
Publicação	NeurIPS 2022 (Datasets and Benchmarks Track)
Total de instâncias	1.000.000 de aplicações individuais
Número de features	30 features (numéricas e categóricas)
Variável alvo	<code>fraud_bool</code> (0 = legítimo, 1 = fraude)
Classes	2 (classificação binária)
Taxa de fraude	$\approx 1,1\%$ (altamente desbalanceado)
Período temporal	8 meses (fevereiro a setembro)
Atributos protegidos	<code>customer_age</code> , <code>income</code> , <code>employment_status</code>

Tabela 4: Metadados do dataset BAF Base utilizado nos experimentos

Cada instância (linha) do dataset representa uma aplicação individual de abertura de conta bancária realizada em uma plataforma online. O rótulo de cada instância é armazenado na coluna `fraud_bool`: uma instância positiva representa uma tentativa fraudulenta, enquanto uma instância negativa representa uma aplicação legítima.

5.3 Descrição das Features

O dataset contém 30 features que representam propriedades observadas das aplicações de abertura de conta. Estas features podem ser obtidas diretamente do solicitante (ex.: status de emprego), derivadas das informações fornecidas (ex.: se o número de telefone é válido), ou calculadas como agregações dos dados (ex.: frequência de aplicações em um dado CEP). As features estão organizadas nas categorias apresentadas na Tabela 5.

Feature	Descrição	Tipo
Informações do Solicitante		
income	Faixa de renda do solicitante (categorizada)	categ.
customer_age	Faixa etária do solicitante (categorizada)	categ.
employment_status	Status de emprego do solicitante	categ.
housing_status	Status de moradia do solicitante	categ.
Informações da Aplicação		
payment_type	Tipo de pagamento selecionado	categ.
intended_balcon_amount	Valor pretendido de saldo/crédito	float
proposed_credit_limit	Limite de crédito proposto	float
days_since_request	Dias desde a solicitação	float
source	Fonte/canal da aplicação	categ.
device_os	Sistema operacional do dispositivo	categ.
Verificações e Validações		
email_is_free	Se o e-mail é de provedor gratuito (0/1)	int
phone_home_valid	Se o telefone residencial é válido (0/1)	int
phone_mobile_valid	Se o telefone celular é válido (0/1)	int
has_other_cards	Se possui outros cartões (0/1)	int
foreign_request	Se a requisição é de origem estrangeira (0/1)	int
name_email_similarity	Similaridade entre nome e e-mail	float
Features de Agregação e Velocidade		
velocity_6h	Velocidade de aplicações nas últimas 6 horas	float
velocity_24h	Velocidade de aplicações nas últimas 24 horas	float
velocity_4w	Velocidade de aplicações nas últimas 4 semanas	float
zip_count_4w	Contagem de aplicações no CEP nas últimas 4 semanas	int
bank_branch_count_8w	Contagem de agências bancárias nas últimas 8 semanas	int
date_of_birth_distinct_emails_4w	E-mails distintos com mesma data de nascimento (4 semanas)	int
device_distinct_emails_8w	E-mails distintos no mesmo dispositivo (8 semanas)	int
device_fraud_count	Contagem de fraudes no dispositivo	int
Histórico do Cliente		
credit_risk_score	Score de risco de crédito	float
bank_months_count	Meses de relacionamento com o banco	int
prev_address_months_count	Meses no endereço anterior	int
current_address_months_count	Meses no endereço atual	int
session_length_in_minutes	Duração da sessão em minutos	float
keep_alive_session	Se a sessão foi mantida ativa	int
Metadados Temporais		
month	Mês da aplicação (0–7, fevereiro a setembro)	int

Tabela 5: Features do dataset BAF

As features capturam uma variedade ampla de informações relevantes para detecção de fraudes: dados do solicitante, informações da aplicação, verificações de validade, métricas de velocidade (que detectam atividade suspeita em janelas temporais), e histórico do cliente. Valores ausentes em certas features são codificados como -1 no dataset.

5.4 Desbalanceamento de Classes

Uma característica fundamental do dataset BAF é o severo desbalanceamento de classes, característico de cenários reais de detecção de fraudes. A taxa de fraude varia entre 0,85% e 1,5% das instâncias ao longo dos diferentes meses, com valores mais altos nos meses posteriores [?]. Este desbalanceamento implica que aproximadamente 98,9% das instâncias são aplicações legítimas e apenas 1,1% são fraudulentas.

Este cenário de desbalanceamento extremo apresenta desafios específicos para o treinamento de modelos de ML:

- **Métricas tradicionais são insuficientes:** Acurácia simples é enganosa, pois um modelo que classifica todas as instâncias como legítimas alcançaria $\approx 98,9\%$ de acurácia.
- **Métrica de produção:** Conforme proposto pelos autores do BAF [?], a métrica principal é o TPR (*True Positive Rate*) no ponto de operação de 5% de FPR (*False Positive Rate*), que é tipicamente imposta por clientes no domínio de detecção de fraudes, pois equilibra a detecção de fraude com a manutenção de baixo atrito para clientes legítimos.
- **Impacto no Aprendizado Federado:** O desbalanceamento deve ser tratado de forma consistente entre os clientes. No particionamento IID balanceado adotado neste trabalho, cada cliente recebe a mesma proporção de classes do dataset original, e o parâmetro `scale_pos_weight` é utilizado para compensar o desbalanceamento durante o treinamento local.

5.5 Estratégia de Divisão Temporal

O dataset BAF utiliza uma estratégia de divisão temporal, aproveitando a coluna `month` que identifica o mês de cada aplicação (0 a 7, correspondendo a fevereiro a setembro). No artigo original [?], os autores adotam uma divisão em 2 conjuntos: treino (meses 0–5) e teste (meses 6–7), apontando a adição de um conjunto de validação como extensão futura. Neste trabalho, estendemos a divisão para 3 conjuntos, seguindo essa recomendação:

- **Treino:** Primeiros 6 meses de dados (meses 0–5, fevereiro a julho), com aproximadamente 600.000 amostras
- **Validação:** Mês 6 (agosto), com aproximadamente 150.000 amostras, utilizado para ajuste de hiperparâmetros e seleção de limiar de decisão
- **Teste:** Mês 7 (setembro), com aproximadamente 150.000 amostras, utilizado exclusivamente para avaliação final do modelo

Esta divisão temporal é preferida à divisão aleatória porque dados mais recentes tendem a ser mais fiéis à distribuição dos dados quando modelos são colocados em produção. Além disso, esta abordagem captura a dinâmica temporal inerente ao domínio de fraudes, onde fraudadores adaptam seus comportamentos ao longo do tempo para evadir detecção — um fenômeno conhecido como *adversarial classification* [?].

5.6 Pré-processamento

O pré-processamento dos dados do BAF para os experimentos de Aprendizado Federado segue um pipeline estruturado implementado na classe `DataPreprocessor` do módulo `data/processing.py`.

5.6.1 Limpeza dos Dados

A primeira etapa do pipeline consiste na limpeza dos dados brutos. Valores codificados como `-1` no dataset original são substituídos por `NaN` para permitir tratamento adequado de valores ausentes. Em seguida, são removidas as colunas `prev_address_months_count`, `intended_balcon_amount` e `device_fraud_count`, identificadas como pouco informativas ou potencialmente problemáticas. Linhas com valores ausentes são descartadas, e a coluna `velocity_24h` é removida por alta correlação com outras features de velocidade. Os valores `NaN` remanescentes nas features numéricas são preenchidos com a mediana calculada exclusivamente nos dados de treino.

5.6.2 Engenharia de Features

Para melhorar a capacidade preditiva dos modelos, são criadas mais de 20 novas features derivadas dos dados originais. Entre as principais, destacam-se: codificação por frequência de `device_os` e `source`; razões de velocidade entre janelas temporais (`velocity_6h / velocity_4w`); Z-scores mensais para features de velocidade e contagem; interações entre idade e renda (`income × customer_age`); categorização de faixa etária em 3 grupos (jovem, adulto, sênior); combinações de status de emprego com quartil de

renda; e transformações logarítmicas de features com distribuição assimétrica. Também é criada uma flag binária `age_above_50` utilizada posteriormente na avaliação de *fairness*.

5.6.3 Codificação e Preparação Final

A coluna `fraud_bool` é extraída como variável alvo (y), e a coluna `month` é utilizada exclusivamente para a divisão temporal. As features categóricas — incluindo as originais (`payment_type`, `employment_status`, `housing_status`, `source`, `device_os`) e as criadas na engenharia de features (`age_group`, `employment_income_cat`, `credit_limit_bucket`) — são codificadas utilizando *One-Hot Encoding* com o parâmetro `drop='if_binary'`, ajustado apenas nos dados de treino e aplicado aos conjuntos de validação e teste. As features numéricas são utilizadas em suas escalas originais, sem normalização adicional, uma vez que modelos baseados em árvore são invariantes a transformações monotônicas nas features.

5.7 Particionamento dos Dados para Federated Learning

5.7.1 Estratégia de Particionamento IID Balanceado

O particionamento dos dados de treino entre os clientes federados segue uma estratégia IID (*Independent and Identically Distributed*) balanceada, implementada na classe `DataPartitioner`. Nesta abordagem, as amostras fraudulentas e legítimas são separadas e distribuídas uniformemente entre os clientes, garantindo que cada participante receba uma proporção equivalente de ambas as classes. Este balanceamento é importante dado o severo desbalanceamento do dataset BAF, pois evita que algum cliente receba proporcionalmente mais ou menos amostras de fraude que os demais.

O processo consiste em separar os índices das amostras por classe (fraude e legítima), embaralhar aleatoriamente cada grupo, dividir cada grupo em partes iguais correspondentes ao número de clientes, e concatenar as partições de cada classe para formar o conjunto de dados de cada cliente. Desta forma, todos os clientes possuem aproximadamente a mesma taxa de fraude do dataset global.

Características da distribuição para FL:

- **Total de dados de treino:** ≈ 600.000 amostras (meses 0–5)
- **Validação:** ≈ 150.000 amostras (mês 6), conjunto centralizado utilizado pelo servidor
- **Teste:** ≈ 150.000 amostras (mês 7), avaliação final

- **Clientes:** Configurável (padrão: 3 clientes)
- **Desbalanceamento de classes:** Tratado via parâmetro `scale_pos_weight` nos modelos, calculado como a razão entre amostras negativas e positivas

6 CONFIGURAÇÃO EXPERIMENTAL

6.1 Ambiente Computacional

Os experimentos foram executados em ambiente virtualizado WSL2 (*Windows Subsystem for Linux 2*) sobre sistema operacional Windows 11, utilizando Python 3.12.3 e o framework de simulação do Flower [?] com backend Ray para orquestração dos clientes federados. A Tabela 6 apresenta as especificações do ambiente.

Componente	Especificação
Sistema Operacional	WSL2 (Ubuntu) sobre Windows 11
Plataforma	Linux 5.15.167.4-microsoft-standard-WSL2 x86_64
Python	3.12.3 (GCC 13.3.0)
Framework FL	Flower (modo simulação com Ray)
Modelos	XGBoost, LightGBM, CatBoost
Otimização	Optuna (busca de hiperparâmetros)

Tabela 6: Especificações do ambiente computacional

O modo de simulação do Flower permite executar servidor e múltiplos clientes em um único processo, utilizando Ray para gerenciar os recursos computacionais e simular a comunicação entre participantes. Esta abordagem é recomendada para prototipagem e avaliação experimental, eliminando a necessidade de infraestrutura de rede real enquanto preserva o comportamento do protocolo federado [?].

6.2 Configuração do Dataset

O dataset BAF Base [?] foi preparado conforme descrito na Seção 5, resultando na configuração apresentada na Tabela 7.

Parâmetro	Valor
Total de amostras	999.641
Número de features	99 (após engenharia de features e OHE)
Conjunto de treino	794.680 amostras (meses 0–5)
Conjunto de validação	108.132 amostras (mês 6)
Conjunto de teste	96.829 amostras (mês 7)
Taxa de fraude (treino)	1,03%
Taxa de fraude (validação)	1,34%
Taxa de fraude (teste)	1,47%
<code>scale_pos_weight</code>	96,53

Tabela 7: Configuração do dataset para os experimentos

O parâmetro `scale_pos_weight`, calculado como a razão entre amostras negativas e positivas no conjunto de treino, é utilizado por todos os três algoritmos para compensar

o severo desbalanceamento de classes durante o treinamento, atribuindo maior peso às amostras da classe minoritária (fraude).

6.3 Otimização Federada de Hiperparâmetros

Os hiperparâmetros de cada algoritmo foram otimizados de forma federada: cada cliente executa independentemente 15 *trials* de busca bayesiana via Optuna, utilizando 33% dos seus dados locais. Os resultados são agregados no servidor via mediana — mais robusta contra *outliers* que a média aritmética — preservando a localidade dos dados. A métrica otimizada é o TPR no ponto de operação de 5% de FPR, com taxa de aprendizado limitada a 0,05 (*safety brake*) para evitar sobreajuste no treinamento federado com *warm start*. Os hiperparâmetros selecionados são apresentados na Tabela 8.

Algoritmo	Hiperparâmetro	Valor
6*XGBoost	max_depth	3
	learning_rate	0,0500
	n_estimators	140
	min_child_weight	7
	subsample	0,9306
	colsample_bytree	0,8955
7*LightGBM	max_depth	4
	learning_rate	0,0270
	n_estimators	151
	num_leaves	61
	min_child_samples	14
	subsample	0,6799
	colsample_bytree	0,8057
4*CateBoost	depth	4
	learning_rate	0,0332
	iterations	230
	l2_leaf_reg	$8,50 \times 10^{-7}$

Tabela 8: Hiperparâmetros otimizados para cada algoritmo

6.4 Configuração do Aprendizado Federado

A Tabela 9 apresenta os parâmetros da simulação federada.

Parâmetro	Valor
Número de clientes	3
Rodadas de comunicação	15
Épocas locais por rodada	15
Fração de amostragem	1,0 (todos os clientes participam)
Particionamento	IID balanceado
Semente aleatória	42
FPR alvo	5%
<i>Learning rate decay</i> (Cíclica)	$0,85^{r-1}$ por rodada r

Tabela 9: Parâmetros da simulação federada

Os dados de treino foram distribuídos uniformemente entre 3 clientes, cada um recebendo aproximadamente 264.893 amostras com 2.716 instâncias de fraude (taxa de 1,03%), preservando a distribuição de classes do dataset original. Os conjuntos de validação e teste são mantidos centralizados no servidor para avaliação global do modelo.

Ambas as estratégias utilizam *warm start*: na Bagging, o melhor modelo da rodada anterior é enviado para todos os clientes; na Cíclica, o modelo é passado sequencialmente entre clientes. A estratégia Cíclica aplica adicionalmente *learning rate decay* com fator 0,85 por rodada para controlar o sobreajuste do treinamento sequencial.

6.5 Experimentos Realizados

Foram conduzidas 6 simulações, correspondendo à combinação de 3 algoritmos (XGBoost, LightGBM e CatBoost) com 2 estratégias de agregação (Bagging e Cíclica), conforme apresentado na Tabela 10.

Algoritmo	Estratégia	Descrição
XGBoost	Bagging	3 clientes treinam em paralelo; árvores agregadas por ensemble
XGBoost	Cíclica	Clientes treinam sequencialmente; modelo passa entre clientes
LightGBM	Bagging	3 clientes treinam em paralelo; árvores agregadas por ensemble
LightGBM	Cíclica	Clientes treinam sequencialmente; modelo passa entre clientes
CatBoost	Bagging	3 clientes treinam em paralelo; árvores agregadas por ensemble
CatBoost	Cíclica	Clientes treinam sequencialmente; modelo passa entre clientes

Tabela 10: Experimentos realizados

Cada simulação executa 15 rodadas de comunicação, e ao final de cada rodada o

modelo global é avaliado no conjunto de validação utilizando dois regimes de limiar:

- **Standard:** Um único limiar global calibrado para 5% de FPR no conjunto completo.
- **Fair:** Limiares independentes por grupo demográfico (idade ≥ 50 e idade < 50), cada um calibrado para 5% de FPR no respectivo grupo. Esta técnica de *post-processing* ajusta os pontos de operação separadamente para cada grupo, equalizando as taxas de falso positivo sem modificar o modelo treinado.

6.6 Métricas de Avaliação

As métricas reportadas para cada experimento são:

- **TPR @ 5% FPR:** Taxa de verdadeiros positivos no ponto de operação de 5% de taxa de falsos positivos, conforme utilizado no artigo do BAF [?] como métrica principal de desempenho em produção.
- **AUC-ROC:** Área sob a curva ROC, medida de discriminação independente do limiar.
- **FPR Ratio:** Razão entre as taxas de falso positivo dos grupos demográficos (min / max), onde 1,0 indica fairness perfeita.
- **Tempo de execução:** Duração total da simulação em segundos.
- **Bytes transferidos:** Volume total de dados transferidos entre clientes e servidor.

O tempo total do experimento completo (incluindo pré-processamento, otimização de hiperparâmetros e as 6 simulações) foi de 7.021 segundos (aproximadamente 1 hora e 57 minutos).

7 RESULTADOS

7.1 Baseline Centralizado

O artigo original do BAF [?] avaliou 100 configurações de hiperparâmetros de LightGBM no dataset Base utilizando treinamento centralizado. Os melhores modelos alcançaram TPR entre 40% e 55% no ponto de operação de 5% FPR, com FPR Ratio em torno de 0,3 — indicando que aplicações legítimas de indivíduos mais velhos tinham mais de três vezes mais chance de serem incorretamente classificadas como fraudulentas. Estes resultados servem como *baseline* comparativo para os experimentos federados.

7.2 Resultados Federados

Os experimentos federados foram conduzidos com 3 clientes em particionamento IID balanceado, 15 rodadas de comunicação e 15 épocas locais por rodada. A Tabela 11 apresenta os resultados finais de todas as configurações, avaliados no conjunto de teste (mês 7) com ambos os regimes de limiar.

Estratégia	Modelo	TPR Std	AUC	FPR Ratio Std	TPR Fair	FPR Ratio Fair
3*Bagging	XGBoost	54,48%	0,8844	0,313	52,94%	0,987
	LightGBM	53,50%	0,8798	0,289	51,19%	0,921
	CatBoost	55,74%	0,8794	0,286	53,71%	0,992
3*Cíclica	XGBoost	56,37%	0,8921	0,284	54,27%	0,999
	LightGBM	55,81%	0,8903	0,288	53,57%	0,912
	CatBoost	56,23%	0,8905	0,293	54,69%	0,995

Tabela 11: Resultados finais no conjunto de teste (mês 7). FPR Ratio = min / max das FPR entre grupos demográficos; 1,0 indica fairness perfeita. Melhores valores em negrito.

O XGBoost Cíclico alcançou o melhor TPR absoluto (56,37%) e AUC-ROC (0,8921), seguido de perto pelo CatBoost Cíclico (56,23%, AUC 0,8905). Na estratégia Bagging, o CatBoost obteve o melhor resultado (55,74%), enquanto o LightGBM ficou abaixo dos demais (53,50%). A estratégia Cíclica superou a Bagging para todos os algoritmos, beneficiando-se do treinamento sequencial com *warm start* e *learning rate decay*. Os valores de AUC-ROC foram similares entre as configurações Cíclicas (0,890–0,892), indicando capacidade discriminativa comparável, com diferença mais acentuada na Bagging (0,879–0,884). A Figura 7 apresenta a comparação visual dos valores de AUC-ROC.



Figura 7: AUC-ROC por modelo e estratégia. Valores calculados no conjunto de teste (mês 7).

No regime standard, todos os modelos apresentaram FPR Ratio entre 0,284 e 0,313, consistente com o *baseline* centralizado ($\approx 0,3$). No regime fair, os FPR Ratios sobem para 0,912–0,999, equalizando as taxas de falso positivo entre grupos com perda média de 1,8 p.p. em TPR. Destaca-se o XGBoost Cíclico, que no regime fair alcançou FPR Ratio de 0,999 (fairness virtualmente perfeita) com TPR de 54,27%, e o CatBoost Cíclico com o melhor TPR fair (54,69%) e FPR Ratio de 0,995. A Figura 8 ilustra o *trade-off* entre performance e equidade para todas as configurações.



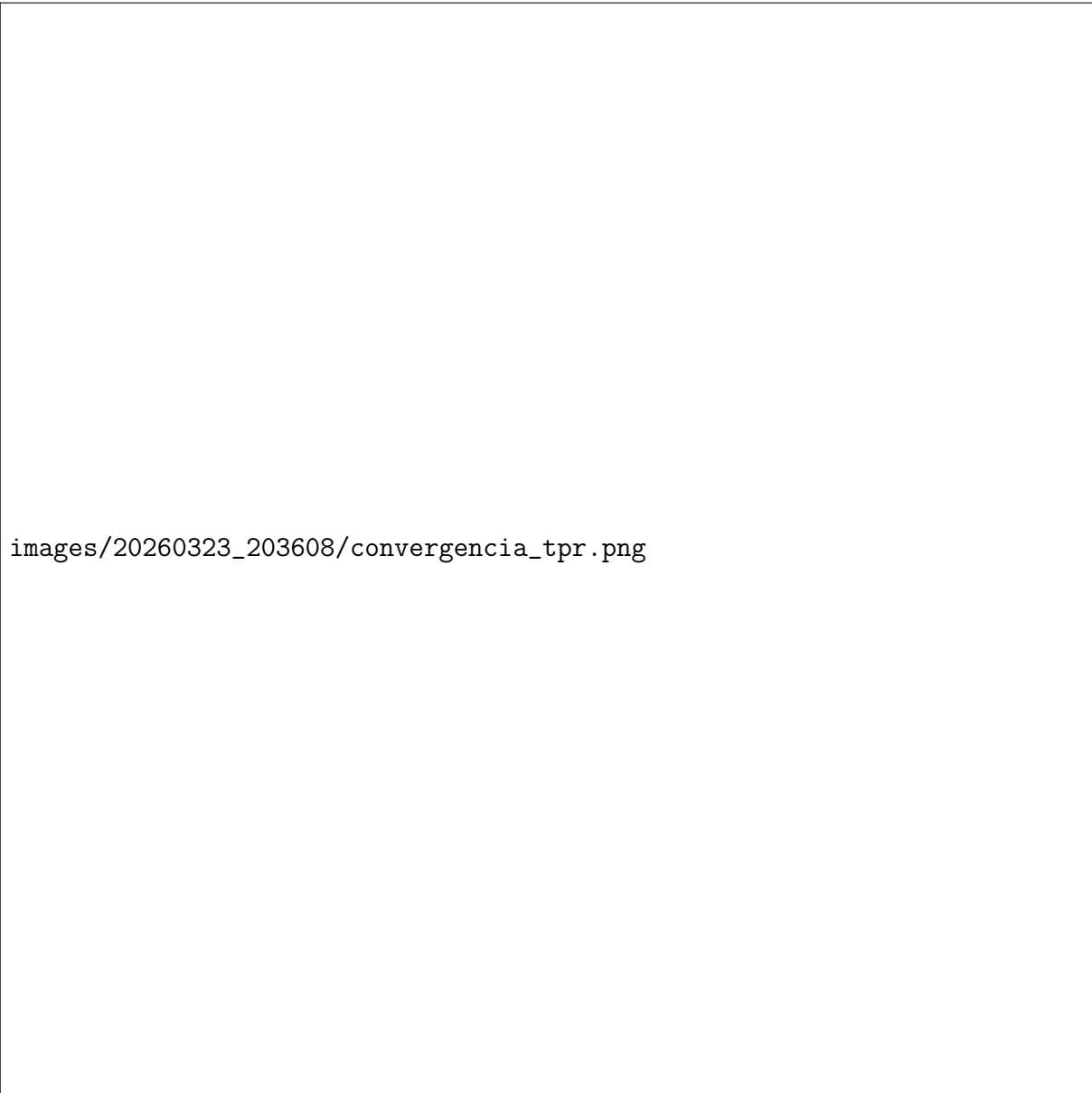
Figura 8: *Trade-off* entre performance (TPR) e equidade (FPR Ratio). Círculos: limiar único; quadrados: limiar por grupo. A região verde indica fairness aceitável ($\geq 0,9$).

7.3 Convergência

A Tabela 12 apresenta a evolução do TPR (regime standard) no conjunto de validação ao longo de rodadas selecionadas, e a Figura 9 mostra a curva completa de convergência.

Estratégia	Modelo	R1	R3	R5	R7	R10	R15
3*Bagging	XGBoost	49,83%	52,38%	52,59%	52,93%	51,97%	52,38%
	LightGBM	50,17%	50,86%	52,04%	52,31%	51,69%	51,83%
	CatBoost	50,17%	51,21%	51,69%	51,97%	51,69%	51,62%
3*Cíclica	XGBoost	49,28%	51,69%	51,97%	51,62%	51,62%	51,76%
	LightGBM	49,21%	50,86%	50,72%	50,79%	50,45%	50,38%
	CatBoost	50,93%	52,24%	51,90%	52,04%	52,04%	51,97%

Tabela 12: Evolução do TPR (limiar standard) no conjunto de validação por rodada selecionada



images/20260323_203608/convergencia_tpr.png

Figura 9: Convergência do TPR@5%FPR ao longo das 15 rodadas. Painel esquerdo: limiar único. Painel direito: limiar por grupo. Linha pontilhada vermelha: *baseline* centralizado (55%).

É importante notar que os valores de TPR por rodada na Tabela 12 (49–53%) são sistematicamente inferiores aos resultados finais da Tabela 11 (53–56%). Isto ocorre porque as métricas por rodada são calculadas no **conjunto de validação** (mês 6, 108.132 amostras) durante o treinamento, enquanto a avaliação final é realizada no **conjunto de teste** (mês 7, 96.829 amostras). Como o limiar de decisão é recalibrado independentemente em cada conjunto para atingir exatamente 5% de FPR, e as distribuições de fraude diferem entre meses (taxa de fraude de 1,34% na validação vs. 1,47% no teste), o ponto de operação resultante produz valores de TPR distintos. Este é o comportamento esperado em divisões temporais, onde meses diferentes possuem padrões de fraude ligeiramente

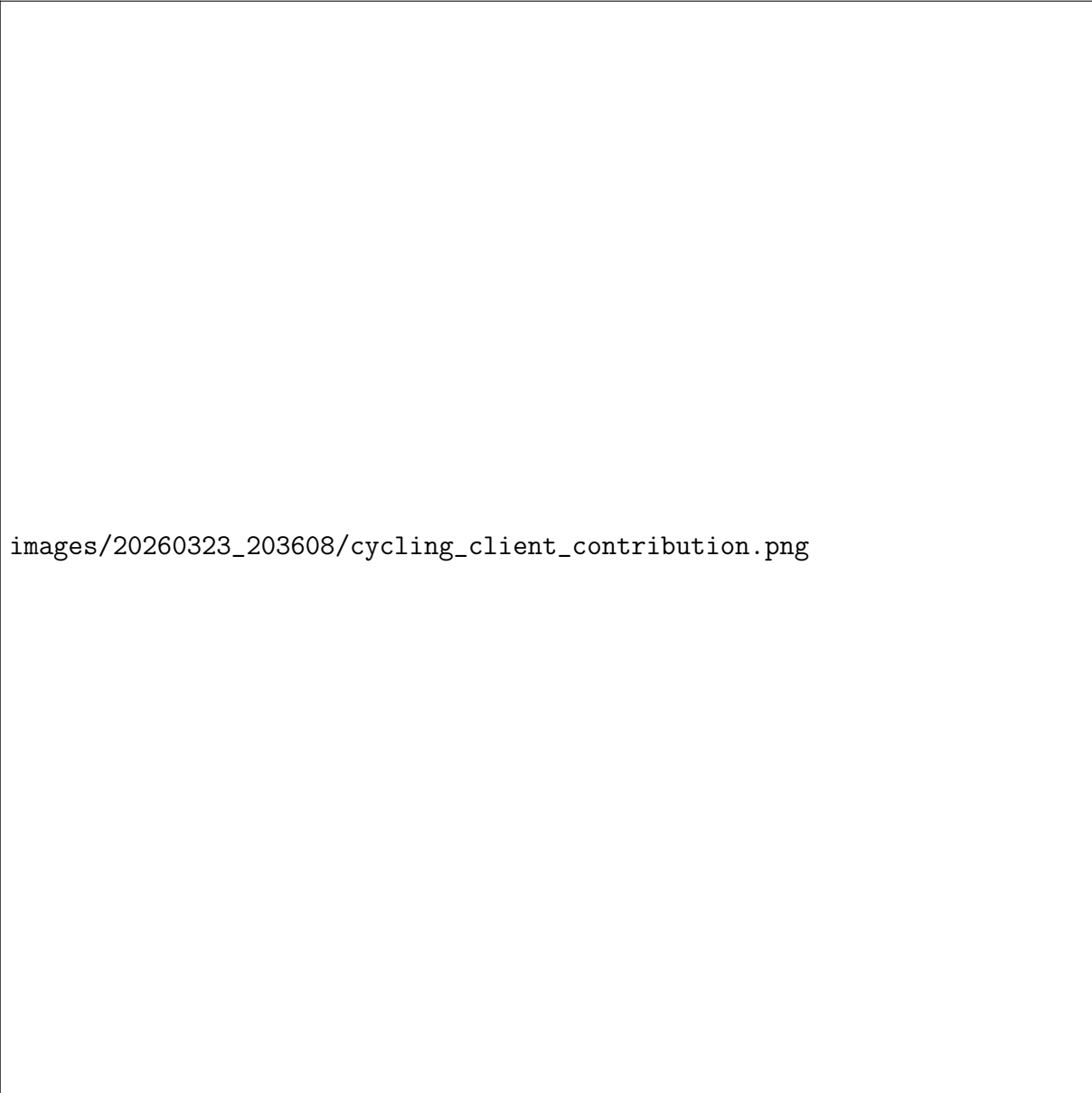
diferentes.

A análise de convergência revela três padrões:

1. **Convergência rápida:** Todos os modelos atingem $\geq 90\%$ do seu desempenho final nas primeiras 3–5 rodadas. O ganho entre a rodada 1 e a rodada 5 é de 2–3 p.p., enquanto o ganho entre as rodadas 5 e 15 é inferior a 0,5 p.p.
2. **Estagnação e leve degradação:** Após atingir o pico (geralmente entre as rodadas 5–7), os modelos estabilizam ou apresentam leve declínio. O XGBoost Bagging, por exemplo, atinge seu pico na rodada 7 (52,93%) e recua para 52,38% na rodada 15. Este comportamento é esperado com *warm start*, pois cada rodada adiciona árvores ao modelo existente, aumentando sua complexidade e o risco de sobreajuste.
3. **Cycling mais estável que Bagging nas rodadas tardias:** A estratégia Cíclica, graças ao *learning rate decay* ($0,85^{r-1}$), apresenta menos oscilação nas rodadas finais. O CatBoost Cíclico mantém TPR estável entre 51,90% e 52,24% da rodada 3 à 15, confirmando a eficácia do controle de taxa de aprendizado.

7.4 Contribuição Sequencial dos Clientes (Estratégia Cíclica)

Na estratégia Cíclica, os clientes treinam em ordem circular (Cliente $0 \rightarrow 1 \rightarrow 2 \rightarrow 0 \rightarrow \dots$), cada um refinando o modelo recebido do cliente anterior. A Figura 10 mostra a contribuição individual de cada cliente ao longo das rodadas.



images/20260323_203608/cycling_client_contribution.png

Figura 10: Estratégia Cíclica: contribuição sequencial de cada cliente. A cor indica qual cliente treinou em cada rodada. A linha cinza conecta os pontos para mostrar a tendência.

Observa-se que a contribuição dos clientes é relativamente uniforme, o que é esperado no particionamento IID balanceado onde todos os clientes possuem a mesma distribuição de dados (1,03% de fraude). Os maiores ganhos ocorrem nos primeiros ciclos completos (rodadas 1–3), onde cada cliente contribui com informação inédita ao modelo. Nas rodadas posteriores, o *learning rate decay* reduz progressivamente a magnitude das atualizações, estabilizando o modelo.

7.5 Eficiência

A Tabela 13 apresenta o tempo de execução e volume de dados transferidos entre clientes e servidor, e as Figuras 11 e 12 ilustram estas métricas.

Estratégia	Modelo	Tempo (s)	Dados Transferidos (MB)
3*Bagging	XGBoost	522,80	55,93
	LightGBM	1.238,16	95,92
	CatBoost	574,85	18,36
3*Cíclica	XGBoost	205,49	18,72
	LightGBM	204,94	32,16
	CatBoost	347,88	9,62

Tabela 13: Tempo de execução e volume de comunicação das simulações federadas (15 rodadas)



Figura 11: Tempo de execução por modelo e estratégia. A estratégia Cíclica é consistentemente mais rápida.



Figura 12: Volume total de dados transferidos por modelo e estratégia. O CatBoost Cíclico é o mais eficiente (9,62 MB).

A estratégia Cíclica é consistentemente mais rápida e econômica que a Bagging para todos os algoritmos, pois treina apenas um cliente por rodada contra três na Bagging. O LightGBM Bagging é o mais lento (1.238s, $\approx$ 21 minutos) e transfere o maior volume de dados (95,92 MB), enquanto o CatBoost Cíclico é o mais eficiente em comunicação (9,62 MB) — uma diferença de $10\times$ que pode ser determinante em cenários com largura de banda limitada. O XGBoost e LightGBM Cíclicos apresentam tempos similares ($\approx$ 205s), demonstrando que a estratégia Cíclica equaliza o tempo de execução entre algoritmos de complexidade similar.

O crescimento do volume de comunicação por rodada é linear: cada rodada transmite o modelo atualizado, que acumula árvores adicionadas pelo *warm start*. No XGBoost Cíclico, o modelo cresce de 167 KB na rodada 1 para 2,45 MB na rodada 15, refletindo o acúmulo de 140 árvores por rodada. Esta propriedade dos modelos baseados em árvore contrasta com redes neurais, cujo tamanho de modelo é fixo independentemente do número de rodadas.

7.6 Comparação Federado vs. Centralizado

Os modelos federados alcançaram TPR entre 53,50% e 56,37%, na faixa superior do *baseline* centralizado do BAF (40–55%). Contudo, a comparação direta é limitada pois os experimentos federados utilizam 99 features (após engenharia de features e OHE), enquanto o *baseline* usa as 30 features originais. Apesar desta diferença, os valores de FPR Ratio no regime standard (0,284–0,313) são consistentes com o *baseline* ($\approx 0,3$), indicando que a federalização não introduz viés demográfico adicional. O CatBoost e o XGBoost, não avaliados no artigo original do BAF, demonstraram desempenho competitivo ou superior ao LightGBM no cenário federado — representando uma contribuição inédita deste trabalho.

8 DISCUSSÃO

8.1 Interpretação dos Resultados

Os resultados experimentais revelam padrões consistentes que permitem extrair conclusões sobre o comportamento dos modelos de *Gradient Boosting* em ambiente federado.

Comparação entre estratégias. A estratégia Cíclica superou consistentemente a Bagging em todos os algoritmos, com o XGBoost Cíclico alcançando o melhor resultado absoluto (56,37% TPR) e o CatBoost Cíclico o segundo melhor (56,23%). Esta superioridade da Cíclica contrasta com a intuição de que o *ensemble* da Bagging deveria produzir modelos mais robustos. A explicação reside no mecanismo de *warm start*: na Cíclica, cada cliente refina sequencialmente o mesmo modelo, acumulando conhecimento de todos os clientes em um único modelo coeso. Na Bagging, embora o melhor modelo da rodada anterior seja enviado como ponto de partida, cada cliente diverge independentemente e a agregação final (*ensemble* por média de probabilidades) pode diluir padrões específicos aprendidos por clientes individuais. Ambas as estratégias utilizam *warm start*, mas o mecanismo de transferência sequencial da Cíclica, combinado com *learning rate decay* ($0,85^{r-1}$), provou-se mais eficaz para preservar e refinar o conhecimento acumulado.

Desempenho por algoritmo. O XGBoost Cíclico alcançou o melhor TPR (56,37%) e AUC-ROC (0,8921), seguido pelo CatBoost Cíclico (56,23%, AUC 0,8905). O LightGBM, embora com desempenho ligeiramente inferior na Bagging (53,50%), alcançou resultado competitivo na Cíclica (55,81%). A diferença de performance entre Bagging e Cíclica foi mais pronunciada no LightGBM (+2,31 p.p.) do que no XGBoost (+1,89 p.p.) e CatBoost (+0,49 p.p.), sugerindo que o LightGBM é mais sensível à estratégia de agregação. Na análise de eficiência, o CatBoost destacou-se pelo menor volume de comunicação (9,62 MB na Cíclica — 10× menos que o LightGBM Bagging), enquanto o XGBoost Cíclico apresentou o melhor equilíbrio entre performance e tempo de execução.

Convergência e número de rodadas. A análise de convergência revela que 15 rodadas são mais que suficientes: os modelos atingem $\geq 90\%$ do desempenho final nas primeiras 3–5 rodadas, com ganhos marginais e até leve degradação nas rodadas posteriores. O XGBoost Bagging, por exemplo, atinge pico na rodada 7 (52,93% na validação) e recua para 52,38% na rodada 15, indicando sobreajuste pelo acúmulo excessivo de árvores via *warm start*. O CatBoost Cíclico demonstrou a convergência mais estável, mantendo TPR entre 51,90% e 52,24% da rodada 3 à 15. Na prática, 5–7 rodadas seriam suficientes para a maioria dos cenários, com redução proporcional no tempo de execução e volume de comunicação.

8.2 Comparação com Estado da Arte

Os resultados federados (TPR de 53,50–56,37%) situam-se na faixa superior do *baseline* centralizado reportado pelo BAF (40–55% com 30 features). Contudo, esta comparação deve ser interpretada com cautela por duas razões.

Primeiro, os experimentos federados utilizam 99 features após engenharia de features e *One-Hot Encoding*, enquanto o *baseline* centralizado utiliza as 30 features originais. A engenharia de features — que inclui razões de velocidade, Z-scores mensais, interações entre atributos e codificação por frequência — contribui significativamente para a capacidade preditiva dos modelos e explica, ao menos parcialmente, o desempenho superior observado.

Segundo, os hiperparâmetros dos modelos federados foram otimizados via Optuna de forma federada (15 *trials* por cliente, agregação por mediana), enquanto o *baseline* do BAF avalia 100 configurações amostradas aleatoriamente. A otimização bayesiana tende a encontrar configurações superiores em menos avaliações.

Apesar destas diferenças metodológicas, dois achados são relevantes: (i) o treinamento federado não introduz degradação severa de desempenho — os modelos permanecem competitivos mesmo com dados distribuídos entre 3 clientes; (ii) os valores de FPR Ratio no regime standard ($\approx 0,28$ – $0,31$) são consistentes com os do *baseline* centralizado ($\approx 0,3$), indicando que a federalização não amplifica o viés demográfico.

8.3 Mecanismo de Correção de Fairness

O ajuste de fairness implementado neste trabalho utiliza a técnica de *post-processing* com limiares de decisão específicos por grupo demográfico. O mecanismo funciona da seguinte forma:

1. O modelo federado produz probabilidades de fraude $p(x)$ para cada transação, independentemente do grupo demográfico.
2. No regime **standard**, um único limiar τ é calibrado no conjunto completo para que a taxa de falsos positivos global seja exatamente 5%. A decisão é $\hat{y} = 1[p(x) \geq \tau]$.
3. No regime **fair**, dois limiares são calibrados independentemente: τ_{jovem} para o grupo idade < 50 e τ_{idoso} para o grupo idade ≥ 50 , cada um ajustado para que o FPR do respectivo grupo seja 5%. A decisão é:

$$\hat{y} = \{ 1[p(x) \geq \tau_{jovem}]_{seidade < 50} 1[p(x) \geq \tau_{idoso}]_{seidade \geq 50}$$

Esta abordagem equaliza as taxas de falso positivo entre grupos sem modificar o modelo treinado, preservando toda a informação aprendida durante o treinamento federado. O FPR Ratio resultante (razão min / max das FPR entre grupos) mede a equidade: valores próximos a 1,0 indicam que ambos os grupos sofrem taxas de falso positivo similares. O custo desta equalização é uma redução no TPR global, pois o limiar do grupo com menor FPR original (tipicamente o grupo jovem) é relaxado, enquanto o do grupo com maior FPR (idosos) é enrijecido. Na prática, esta perda foi de 1,5–2,3 p.p. em TPR, um custo moderado para garantir que clientes de diferentes faixas etárias não sejam desproporcionalmente afetados por falsos alertas.

8.4 Trade-off entre Fairness e Desempenho

A análise dos dois regimes de limiar revela um *trade-off* controlável entre desempenho e fairness, claramente visível na Figura 8. No gráfico, os pontos com limiar único (círculos) concentram-se no canto superior esquerdo — alto TPR mas baixo FPR Ratio ($\approx 0,29$) — enquanto os pontos com limiar por grupo (quadrados) migram para o canto superior direito — TPR ligeiramente menor mas FPR Ratio próximo de 1,0.

O regime fair eleva o FPR Ratio de $\approx 0,28$ – $0,31$ para $\approx 0,91$ – $1,00$, reduzindo significativamente a disparidade nas taxas de falso positivo entre grupos etários. O custo em TPR é moderado: uma perda média de 1,8 pontos percentuais, variando de 1,54 p.p. (XGBoost Bagging) a 2,31 p.p. (LightGBM Bagging). Destaca-se o CatBoost Cíclico, que no regime fair alcançou o melhor TPR (54,69%) com FPR Ratio de 0,995 — o melhor equilíbrio geral entre performance e equidade.

Este resultado tem implicação prática direta: instituições financeiras podem adotar o regime fair para cumprir requisitos regulatórios de não-discriminação [?] com impacto limitado na capacidade de detecção de fraude. A perda de ≈ 2 p.p. em TPR é uma troca aceitável para garantir que clientes legítimos de diferentes faixas etárias não sejam desproporcionalmente afetados por falsos alertas de fraude.

8.5 Desafios de Implementação

Durante o desenvolvimento e validação dos experimentos, foram identificados e corrigidos problemas que impactavam diretamente a confiabilidade dos resultados.

Implementação do *warm start* para Bagging. A estratégia Bagging originalmente treinava todos os clientes do zero em cada rodada. Foi implementado um mecanismo onde o servidor seleciona o melhor modelo da rodada anterior (baseado no TPR de validação) e o envia como ponto de partida para todos os clientes na rodada seguinte. Esta modificação resultou em convergência mais rápida e ganhos progressivos a cada rodada.

Controle de sobreajuste na estratégia Cíclica. Observou-se que a Cíclica, por treinar sequencialmente o mesmo modelo em diferentes clientes, tendia a sobreajustar nas rodadas posteriores — o TPR de validação declinava após atingir um pico nas rodadas intermediárias. Para mitigar este efeito, foi implementado um *learning rate decay* exponencial com fator $0,85^{r-1}$, onde r é o número da rodada. O *decay* reduz progressivamente a taxa de aprendizado (por exemplo, de 0,05 na rodada 1 para 0,005 na rodada 15), permitindo ajustes mais finos nas rodadas avançadas e estabilizando a convergência.

Tratamento de valores faltantes e engenharia de features. O dataset BAF utiliza o valor -1 como indicador de dados faltantes em diversas colunas. Foi necessário substituir esses valores por NaN e preencher com a mediana de cada coluna, resultando na remoção de aproximadamente 359 amostras com valores críticos ausentes. A expansão de 30 para 99 features através de engenharia de features — incluindo codificação por frequência, Z-scores mensais e interações entre atributos — exigiu cuidado para manter a consistência entre os conjuntos de treinamento, validação e teste. O *One-Hot Encoding* foi ajustado com `handle_unknown='ignore'` para evitar erros quando categorias presentes no teste não apareciam no treinamento.

Desbalanceamento severo de classes. A taxa de fraude no conjunto de treinamento é de apenas 1,03%, resultando em um `scale_pos_weight` de 96,53. A solução adotada foi a atribuição de pesos de classe nativos de cada framework (`scale_pos_weight` para XGBoost e LightGBM, `auto_class_weights='Balanced'` para CatBoost), que atribuem maior importância às amostras da classe minoritária durante o treinamento. Esta abordagem é compatível com o particionamento federado, pois não requer acesso a dados de outros clientes, diferentemente de técnicas de sobreamostragem como SMOTE.

Serialização heterogênea de modelos. A transmissão de modelos entre clientes e servidor requer serialização em bytes, porém cada framework possui uma API distinta: o XGBoost permite serialização direta em JSON via `save_raw()`, enquanto LightGBM e CatBoost exigem gravação em arquivos temporários. Para unificar o processo e evitar código duplicado, foi adotada uma abordagem baseada em *pickle*, onde todos os modelos são serializados uniformemente como arrays NumPy.

Otimização federada de hiperparâmetros. A otimização de hiperparâmetros em ambiente federado apresenta o desafio de que cada cliente pode encontrar configurações ótimas distintas para seus dados locais. A solução adotada executa o Optuna independentemente em cada cliente (utilizando 33% dos dados locais para viabilizar as 15 tentativas) e agrega os resultados via mediana. Adicionalmente, foi implementado um limite máximo de 0,05 para a taxa de aprendizado (*safety brake*), evitando que configurações excessivamente agressivas sugeridas pelo Optuna causassem instabilidade no treinamento federado.

Crescimento do modelo com *warm start*. O mecanismo de *warm start*, que

permite ao modelo continuar o treinamento a partir do ponto onde o cliente anterior parou, tem como efeito colateral o crescimento linear do modelo. No XGBoost Cíclico, o modelo cresce de 167 KB (rodada 1, 140 árvores) para 2,45 MB (rodada 15, ≈ 2.100 árvores). Este crescimento impacta tanto o volume de comunicação quanto o tempo de inferência, e sugere que em cenários com muitas rodadas pode ser necessário implementar técnicas de poda ou limitação do número total de árvores.

8.6 Limitações do Trabalho

Este trabalho possui limitações que devem ser consideradas na interpretação dos resultados:

1. **Particionamento IID:** Os experimentos utilizam particionamento IID balanceado, que não captura a heterogeneidade estatística presente em cenários reais onde diferentes instituições possuem perfis de clientes distintos [?]. Cenários Non-IID podem degradar significativamente a convergência e o desempenho dos modelos.
2. **Número de clientes:** Apenas 3 clientes foram utilizados nas simulações. Cenários reais podem envolver dezenas ou centenas de participantes, introduzindo desafios adicionais de escalabilidade e comunicação [?].
3. **Engenharia de features:** A engenharia de features e *One-Hot Encoding* expandem o dataset de 30 para 99 features, impedindo comparação direta com o *baseline* de 30 features do BAF.
4. **Execução única:** Os experimentos foram executados uma única vez com semente fixa (*seed=42*), sem repetições para estimativa de variância. Análises estatísticas mais robustas exigiriam múltiplas execuções.
5. **Ausência de mecanismos de privacidade:** O sistema não implementa Agregação Segura [?] ou Privacidade Diferencial [?], focando na avaliação de desempenho dos algoritmos de árvore no contexto federado.
6. **Convergência e excesso de rodadas:** A análise mostrou que os modelos saturam entre as rodadas 5–7, sugerindo que 15 rodadas são excessivas para o cenário IID com 3 clientes. Cenários com mais clientes ou dados heterogêneos podem requerer mais rodadas.

8.7 Implicações Práticas

Os resultados sugerem que modelos de *Gradient Boosting* baseados em árvore são candidatos viáveis para sistemas federados de detecção de fraude bancária, com as seguintes implicações:

- **Escolha de algoritmo:** O XGBoost Cíclico é recomendado para cenários que priorizam desempenho absoluto (56,37% TPR). Para cenários que priorizam eficiência de comunicação, o CatBoost Cíclico oferece TPR competitivo (56,23%) transferindo apenas 9,62 MB em 15 rodadas. Para cenários que priorizam fairness com alta performance, o CatBoost Cíclico alcança TPR fair de 54,69% com FPR Ratio de 0,995.
- **Escolha de estratégia:** A Cíclica superou a Bagging em todos os algoritmos, com vantagens adicionais em tempo de execução e volume de comunicação. A Bagging pode ser preferível em cenários Non-IID, onde o *ensemble* de modelos treinados em distribuições distintas pode oferecer maior robustez.
- **Fairness:** A adoção de limiares por grupo permite equalizar taxas de falso positivo com perda moderada de capacidade de detecção (média de 1,8 p.p.), viabilizando conformidade regulatória.
- **Eficiência:** A baixa transferência de dados dos modelos de árvore (9,6–95,9 MB em 15 rodadas) é compatível com redes de largura de banda limitada, reforçando a vantagem sobre modelos baseados em redes neurais profundas [?, ?]. Com a recomendação prática de 5–7 rodadas, estes volumes seriam reduzidos proporcionalmente.

9 CONCLUSÕES E TRABALHOS FUTUROS

9.1 Conclusões

9.2 Contribuições

9.3 Trabalhos Futuros

REFERÊNCIAS

- [1] AUTOR, A.B.; COAUTOR, C.D. Título do artigo. **Nome da Revista**, v.10, n.2, p.45-62, 2020.
- [2] AUTOR, Nome. **Título do Livro**. Editora, Cidade, Ano.
- [3] AUTOR, A.; COAUTOR, B. Título do artigo. **Nome da Conferência**, p.100-110, 2021.